



**UNIVERSITÀ DEGLI STUDI
DI CAGLIARI**

MSC Electronic Engineering

UNIVERSITY OF CAGLIARI

Pet Tracker

IoT Class Project

Professors:

Michele Nitti
Luigi Serreli

Students:

Marco Fois
(ID: 70/83/65487)

Stefano Concas
(ID: 70/83/65486)

Academic Year: 2025/2026

Contents

1	Problem Overview	2
2	Analysis phase	3
2.1	Reference scenario	3
2.2	As-is systems analysis	4
2.2.1	UWB based Systems	5
2.2.2	GPS based Systems	6
2.2.3	Motes based Systems	7
2.2.4	Summary	8
2.3	System Requirements	8
2.3.1	Functional requirements	8
2.3.2	Non-Functional requirements	11
2.4	To-be system analysis	12
2.4.1	User-friendly interface	12
2.4.2	Vision-based Pet Detection and Localization	12
2.4.3	Higher scalability	12
2.4.4	Easy maintenance	13
2.5	Available hardware	13
2.5.1	Development boards	13
2.5.2	Sensors and actuators	16
2.5.3	Communication technologies and application protocols	20
3	Design phase	25
3.1	On-Field System Architecture	26
3.1.1	Passage Detection Module	26
3.1.2	Visual Acquisition Module	29
3.1.3	Acoustic Deterrence Module	31
3.2	Server	33
3.2.1	Digital Twin Representation	33
3.3	The Flask application	42
3.4	Client	44
3.4.1	Web Application	44
3.4.2	Telegram BOT	50

1 Problem Overview

In a typical home pets move freely between rooms, alternating periods of rest in specific spots and active exploration. During their active phases, they might jump onto desks with electronic devices or important paper documents, enter bedrooms risking ruining personal belongings or knocking over valuable items, investigate the kitchen where they might try to reach exposed food or get too close to appliances , or introduce hygiene issues due to shedding.

Furniture and other household objects can also present hazards, especially when pets navigate environments characterized by poor lighting or fragile items.

Managing pet access to domestic spaces is complex because their actions overlap with sensitive or restricted areas. Often, pets are not yet fully trained to recognize these limits, and an untrained pet will naturally repeat unwanted behaviors; such as wandering into forbidden rooms, playing with household items, or getting into unsafe spots without understanding the rules of the house.

This challenge is further compounded by the fact that owners are not always at home to monitor their pets in real time. When they are away whether due to work commitments or other reasons, they need to be able to keep everything under control and promptly verify the pet's location relative to permitted areas.

Knowing the pet's exact position is therefore crucial to recognize when these training gaps lead to trouble, ensuring that accidents are prevented, items are protected, and daily life can proceed safely even without human supervision.

2 Analysis phase

This chapter describes the reference scenario and the assumptions upon which it is based.

Subsequently, an analysis of existing state-of-the-art systems will be conducted to identify similar solutions; based on their respective categories, their advantages and limitations will be highlighted.

Finally based on this analysis, it will be possible define the functional and non-functional requirements for the new system , which will be proposed at the end of the chapter.

2.1 Reference scenario

In everyday life, monitoring a pet's activities and actions is not always simple or feasible, as it would require significant effort and time constraints. The system is therefore designed for workers, students, and people in general who keep a pet at home and spend part of their day engaged in routine daily activities. The number of users can range from one to several people, though it typically involves the household's residents.

Real-time monitoring of your pet takes place, as you can easily imagine, within the walls of your own home. This means that the system must be specifically designed to operate in indoor environments, where a Wi-Fi connection must be available. It must function efficiently in a wide variety of architectural settings, including multi-story homes, regardless of the number of rooms they contain and under various lighting conditions.

As discussed earlier, monitoring must be available at any time of day in order to prevent unwanted situations, whether residents are present in the home or whether no one is there except the pet itself.

The system must be designed to ensure that the pet, which can be a dog or a cat, does not enter restricted rooms for various reasons, primarily related to the daily routines of the household's residents. If the pet enters a restricted area, the system must intervene by applying a corrective action for training purposes. Furthermore, residents must be able to track their pet's current location in real time when they are unable to monitor it directly, whether because they are away from home or engaged in specific activities for extended periods.

To summarize, the system must be robust and reliable in a wide range of situations, whether or not people are present in the home, under various lighting conditions in the rooms, and regardless of the time of day.

2.2 As-is systems analysis

With the reference scenario clearly defined, it is now possible to analyze existing state of the art systems.

To ensure real-time monitoring of pets, a variety of technologies are used today, which in most cases are combined to achieve optimal results. Most systems have been designed to incorporate minimal hardware that can be worn by the animal itself typically on a collar or harness, so as not to restrict the animal's natural movements.

Furthermore, the tremendous progress in the fields of machine learning, computer vision, and, more generally, artificial intelligence (AI) has led to numerous advantages in terms of the ability to identify specific animal targets and to study the kinematics of their movements. In fact, today these models are also used within tracking systems to monitor pets' activity. Thanks to these capabilities, the system is able to make decisions autonomously and in real time.

Currently, the state of the art solutions that have been implemented can be classified as follows:

- **UWB-based Systems:** Systems designed for ultra high precision indoor positioning, based on Ultra-Wideband technology, ideal for covering the entire home through the use of fixed anchors and lightweight wearable devices.
- **GPS-based Systems:** Solutions designed for real-time tracking via satellite geolocation and mobile networks, equipped with an independent SIM module to autonomously transmit data to the cloud and directly monitor the defined perimeter area.
- **Motes-based Systems:** Wireless sensor networks (WSNs) consisting of a mobile node attached to the animal and stationary nodes distributed throughout the rooms, capable of combining radio signal proximity and accelerometer data for home monitoring.

The following subsections will provide an in-depth analysis of these technologies, each accompanied by its advantages and disadvantages.

2.2.1 UWB based Systems

This type of system is typically designed for indoor tracking of pets and posture detection, representing a significant advancement in the field of “Pet Analytics” [1] a rapidly growing sector driven by the increasing number of pets in households. This type of system stems from pet owners’ need to feel connected to their pets and to monitor their well-being, offering a technological alternative to traditional methods based exclusively on video surveillance.

This type of system is primarily based on *Ultra-Wideband Radio* (UWB) technology, which is used for wireless localization. The operating principle involves measuring the time-of-flight of radio signals between a wearable device which can be attached to a collar but is typically mounted on a harness and fixed points called *anchors* to calculate the position using the trilateration method. The anchors are primarily installed in walls at strategic locations and are designed to respond to signals sent by the wearable device to enable distance calculation. These systems also require a Central Processing Unit, which is essentially a radio device that acts as a passive “listener,” intercepting all messages exchanged between the collar and the anchors. It processes the data, determines the pet’s relative position, and periodically sends this information to a cloud server, making the data available to the system’s owners.

Advantages UWB is chosen for its ability to penetrate walls and furniture and for its extremely high precision, which, in the best cases, reaches levels of accuracy on the order of centimeters. Furthermore, these systems primarily transmit data in the form of coordinates, requiring minimal network traffic. Finally, the radio signals operate independently of the lighting conditions in the rooms, covering the entire home with just a small number of sensors.

Disadvantages In order for this type of system to function properly, the user must configure the dimensioned floor plan of the house, which must be known in perfect detail. In addition, the system also requires knowledge of the exact location of the anchors within the floor plan itself. All this information makes the user interface very complex to configure.

2.2.2 GPS based Systems

Among the most widely used tracking systems today are those based on GPS [2]. This technology was primarily designed to ensure the safety of your four legged friend through real-time tracking. Typically, these systems are integrated into the animal's collar. They consist of a microcontroller that acts as the central processing unit, a GPS module to receive coordinates (latitude and longitude) from satellites, and finally a GSM SIM module to establish an Internet connection. Typically, this type of tracker operates as follows: first, the coordinates detected by the GPS are processed by the microcontroller and sent via the mobile network to a cloud database; then, the user interacts with the system through a dedicated app, which downloads the data and displays the pet's location.

Advantages Tracking devices based on this technology, which include a SIM card, have the advantage of having their own independent internet connection to send data to the cloud. This means that tracking does not depend on the presence of local Wi-Fi networks. Furthermore, the use of a microcontroller integrated into the hardware worn by the pet allows the device to independently determine whether a set of coordinates lies inside or outside a predefined polygonal area created by the user.

Disadvantages This type of tracker is, as one might easily guess, very robust and accurate outdoors, with extremely fast response times (typically a few seconds). However, although they are also widely used indoors, their position-tracking accuracy decreases in such environments, with errors on the order of meters. Furthermore, response times increase significantly compared to outdoor use; in the worst-case scenario, they can even reach several minutes.

2.2.3 Motes based Systems

Another emerging technology used for monitoring pets indoors is based on *mote* devices, also known as “sensor nodes” [3]. These are hardware components designed to form wireless sensor networks (WSNs). These small devices collect data from the environment, process the information, and transmit it wirelessly to other nodes or to a central unit. This technology is capable of capturing complex data regarding the animal’s physical activities and environmental conditions through accelerometers and light sensors.

These systems use a mobile node, which is attached to the pet, while several stationary nodes are placed throughout the house. The stationary nodes transmit the data to a stationary node that is typically connected to a PC via an interface card (gateway) in order to send the collected data for analysis.

The principle of localization is extremely simple. The system uses the proximity of the mobile mote to the stationary motes to determine the animal’s current position by combining this information with data from the accelerometer. Depending on which stationary mote the animal is closest to and which will therefore receive a stronger signal, it’s possible to identify the room the animal is in.

Advantages The main advantage of this type of system lies in its effectiveness in indoor environments. Furthermore, the system does not require the device worn by the animal to be in close proximity to the base station at all times. Thanks to stationary nodes that act as repeaters, it is possible to cover large homes or multi-story buildings simply by adding nodes, without losing the signal.

Disadvantages One of the main limitations of this technology stems from the fact that radio signal strength is not always reliable for determining the animal’s exact location. Most systems that use these sensors must also rely on data from an accelerometer to correct the acquired data, preventing the animal from appearing to move when it is actually stationary. Furthermore, another factor to consider is the complexity of installation. In fact, the system requires the placement of several “nodes” (stationary devices) in various rooms of the house and a base station connected to a PC in order to function. Last but not least, these sensors may be cumbersome for small animals.

2.2.4 Summary

The key features of the technologies mentioned above are summarized in [Table 1](#).

Category	Advantages	Disadvantages
UWB-based Systems	• Extremely high precision, down to the centimeter • Complete coverage of the home with just a few anchors • Minimal network traffic • Independence from room lighting conditions	• Complex User Interface Configuration • Complex and precise positioning and configuration of the anchors • Significant hardware bulk on the animal
GPS-based Systems	• Independent SIM connection • Local Calculation of Virtual Boundaries Using a Microcontroller • Independence from room lighting conditions	• Loss of accuracy indoors • Longer response times compared to other technologies
Motes-based Systems	• Roofing for large or multi-story homes • Independence from room lighting conditions	• Limited reliability of the radio signal • Complexity of System Installation

Table 1: Summary of Advantages and Disadvantages of Pet Tracking Technologies

2.3 System Requirements

An analysis of the current state of the art makes it possible to define the requirements that the new system must meet in order to operate effectively in the outlined scenario. Last but not least, meeting these requirements enables the new solution to compete with existing ones.

2.3.1 Functional requirements

The Functional Requirements are listed below: they describe the features of the new system.

- **FR1 User registration**
 - **Description:** The system must allow a new user to create an account by providing personal credentials, enabling them to access the system subsequently.
 - **Input:** User registration data (username and password).
 - **Output:** Creation and secure storage of a new user profile within the system database.

- **FR2 Authentication**

- **Description:** The system must ensure that only authorized users can operate and receive information within the home virtual environments.
- **Input:** Authentication request including username and password.
- **Output:** If the user is authorized, can operate and receive information within the home virtual environments. If not, it cannot.

- **FR3 Room configuration**

- **Description:** The system must allow the administrator user to manage room profiles within the home environment and set their access status as "allowed" or "restricted".
- **Input:** User interactions through the interface regarding room profiles.
- **Output:** Updated room configurations.

- **FR4 Door configuration**

- **Description:** The system must allow the administrator user to manage door profiles within the home environment.
- **Input:** User interactions through the interface regarding door profiles.
- **Output:** Updated door configurations within the system.

- **FR5 Pet profile configuration**

- **Description:** The system must allow the administrator user to configure a single pet.
- **Input:** User actions through the interface.
- **Output:** Creation and storage of a single active pet profile.

- **FR6 Animal recognition**

- **Description:** The system must be able to discriminate the pet from other entities inside the house. Each data acquired by the sensors must be analyzed by a detection algorithm able to find and verify the presence of the animal.
- **Input:** Environmental data captured within the room.
- **Output:** For each captured input, the result of the recognition is returned, indicating whether the pet has been detected.

- **FR7 Pet localization**

- **Description:** The system must determine which room the pet is currently located in through a two-step verification process. Upon detecting a movement from a room to the adjacent one, the system must verify and confirm the identification of the pet (FR6) before updating its location.
- **Input:** Movement detection signal and environment data captured within the room.
- **Output:** Verified confirmation of the pet's current location (room identifier).

- **FR8 Violation detection and deterrent**

- **Description:** The system must detect when the pet enters a "restricted" room, send a notification to the user, and trigger an automated deterrent signal on the pet's wearable device to encourage it to leave the room.
- **Input:** Estimated current position (output of FR7) evaluated against the map permissions (output of FR3).
- **Output:** Activation command sent to the pet's wearable deterrent device and delivery of an alert notification to the user's device.

- **FR9 Real-time position query**

- **Description:** The system must allow the user to query and view the last known location of the pet in real time through the interface.
- **Input:** The interaction of the user with the application interface.
- **Output:** On-screen display of the room where the pet is currently located.

- **FR10 Statistics and logs management**

- **Description:** The system must record boundary violation events and compute room occupancy statistics over time.
- **Input:** Room transition events and violation records.
- **Output:** Display of occupancy metrics and historical violation logs on user request.

- **FR11 Administrator role assignment**

- **Description:** The system must automatically assign the Administrator role to the user who creates a virtual home environment, giving them full control over layout configuration, pet profile, and viewer authorization.
- **Input:** User action creating a new home environment.
- **Output:** Creation of the home environment and assignment of the Administrator role.

- **FR12 Viewer role management**

- **Description:** The system must allow the administrator to add or remove a registered user as a Viewer, granting or revoking their read-only access to monitor the pet's location and status.
- **Input:** Administrator command selecting a registered user and specifying the action (add or remove).
- **Output:** Association or disassociation of the user as a Viewer for that home environment.

- **FR13 Remote deterrent activation**

- **Description:** The system must allow the administrator user to manually trigger the deterrent signal on the pet's wearable device at any time through the application interface.
- **Input:** User command via the interface.
- **Output:** Manual activation command sent to the pet's wearable deterrent device.

- **FR14 Home device status monitoring and alerting**

- **Description:** The system must monitor the connection state of home devices and notify the user about status changes, such as disconnections affecting single or multiple devices requiring action, full system outages, or complete service recovery.
- **Input:** Device connection status updates.
- **Output:** Status notification sent to the user (single/multiple device disconnection alert, full outage alert, or system restored notice).

2.3.2 Non-Functional requirements

In addition to the functional requirements mentioned above, the system introduces the following non-functional requirements, which are intended to enhance its functionality and overall quality standards.

- **NFR1 Scalability**

- The system should not impose any limit on the number of devices.

- **NFR2 Component replaceability**

- The system must be composed of non-complex and standard components that are easy to replace, meaning a broken component can be removed and replaced with a new one without requiring additional maintenance.

- **NFR3 User-friendly interface**

- The system must have an intuitive and user-friendly interface, enabling pet owners to perform their tasks (like querying the pet's position or occupancy statistics) efficiently and with minimal training.

- **NFR4 Latency**

- The system must detect a restricted room violation and deliver the notification to the user's device within 3 seconds of the event occurrence.

- **NFR5 Privacy**

- To protect the privacy of the domestic environment, access to the pet's location data, history, and system configurations must be restricted. The system must ensure that only the primary user and trusted individuals who possess valid security credentials (username and password) can log in and view the information.

- **NFR6 System robustness**

- The system must implement graceful degradation to ensure operational continuity in the event of a subsystem failure. If a specific module (e.g., the pet localization subsystem) becomes unavailable, the core infrastructure must remain functional, allowing users to authenticate, query historical statistics, and manually trigger the wearable deterrent (unless it is the failing module).

2.4 To-be system analysis

Based on the requirements analysis conducted earlier, the following additional key features that the new system must have been identified.

2.4.1 User-friendly interface

As discussed in the previous sections, managing a home environment and ensuring the pet's safety can be complex tasks that require the system to handle multi-user permissions and immediate notifications.

Therefore, in accordance with **FR1–FR5**, **FR9–FR12**, and finally **FR14**, it is essential to implement a modern, user-friendly interface (**NFR3**) that provides a comprehensive overview of the system while remaining easy to understand, intuitive, and protected against unauthorized access (**NFR5**).

An interface of this type simplifies configuration and monitoring processes by allowing the administrator to define the home's layout without having to configure a dimensioned floor plan of the space, set the relevant access permissions, configure individual pet profiles, and add viewer users, thereby facilitating role management and supporting the owners' daily routines.

Furthermore, in accordance with **FR8**, **FR9**, **FR10**, and **FR14**, both administrators and viewers can view, upon request and in real time, the pet's location and occupancy statistics automatically updated by the system; they can also stay constantly updated on the connection status of the system's components, in addition to allowing the administrator to manually activate the wearable deterrent on the pet (**FR13**): An appropriate interactive procedure is required to efficiently meet these requirements.

2.4.2 Vision-based Pet Detection and Localization

In accordance with **FR6** and **FR7**, the system must be able to recognize and locate the pet inside the house and, specifically, in the room where it is located. To meet these requirements, the system relies on a camera installed in each room. Whenever a specific sensor to which the camera is sensitive detects movement from a room adjacent to the one where the camera is located, the camera takes a picture that is immediately processed by an artificial intelligence computer vision algorithm. This algorithm analyzes the photo to accurately distinguish the pet from any other entity (such as a person, a cleaning robot, or any other moving object) that might have triggered the camera, thereby ensuring targeted detection and eliminating false positives.

2.4.3 Higher scalability

The successful implementation of the new system depends on its ability to adapt to various residential scenarios, including multi-story buildings. Requirements **FR3**, **FR4**, and **NFR1** define as a crucial aspect of this system the ability to support the addition of new rooms, areas, or structural changes over time (such as the introduction of new spaces or reductions in the physical environment) without imposing rigid limits on the infrastructure or the number of devices. Therefore, the system will be designed with full scalability in mind, allowing for the seamless integration of new areas and hardware components as the home's needs evolve.

2.4.4 Easy maintenance

An analysis of previous systems clearly shows that the management and repair of infrastructure based on complex or monolithic devices can prove particularly costly in the event of a failure. Therefore, in accordance with **NFR2**, the principles of fault tolerance described in **NFR6**, and the ability to notify residents of the home if there is a need to intervene on certain devices because they are disconnected from the system (**FR14**), the new architecture has been designed to prioritize the use of separate, standardized hardware components. This approach makes maintenance extremely simple and straightforward: if a single device fails, it is sufficient to remove it and replace it with a new one, without the need for complex interventions or compromising the functioning of the entire home infrastructure.

2.5 Available hardware

Before defining the proposal for the new system in detail, it is essential to examine the technologies currently available, taking into account both hardware and software components as well as communication protocols. In light of the requirements defined above, this analysis is explored in greater depth in the following sections.

2.5.1 Development boards

An overview of the development boards under consideration, along with their respective advantages and disadvantages, is provided in [Table 2](#). Each of these boards could be used to implement the new system; however, given the project requirements, it is necessary to find a good balance between cost and computational performance. A brief description of each board is provided below.

Arduino Uno is the successor to the previous Arduino Duemilanove model and features an ATmega328 microcontroller. In terms of hardware, it has 14 digital I/O pins (6 of which can function as PWM outputs) and 6 analog inputs, while its clock is driven by a 16 MHz crystal oscillator. In terms of connectivity and power, it includes a USB port, a standard jack, an ICSP header, and the classic reset button. It essentially has everything it needs on board to operate independently once the firmware has been flashed: it needs to be connected to a computer via a USB cable (which serves as a power source), or to a battery or an external power supply.

Arduino Nano is an extremely compact development board, specifically designed to be inserted and used directly on breadboards [4]. While offering the same processing power and nearly all the features of larger models (such as the Arduino Uno or Duemilanove, thanks to the use of the ATmega328 microcontroller), it stands out for its small size and the absence of the classic DC power connector. It is commonly used in prototypes, embedded systems, and any electronics projects where space is limited.

Raspberry Pi Zero W is a small, lightweight computer based on ARM architecture, capable of performing many of the functions of a standard desktop PC [5]. The board does not have a built-in hard drive or solid-state drive, but uses an SD card for system boot and long-term data storage.

ESP8266 is a low-cost, compact system-on-a-chip that includes a CPU, RAM, and built-in Wi-Fi connectivity [6]. It is widely used as the main component in Internet of Things (IoT) projects. Its low cost makes it accessible, while its built-in connectivity enables wireless communication.

Board	Strenghts	Limitations	Cost
Arduino Uno	• Extensive documentation and a well-established library ecosystem. • Modular architecture that can be expanded using dedicated Shields.	• Extremely limited computational capacity and memory (SRAM/Flash). • No built-in network or wireless connectivity.	29,30 €
Arduino Nano	• Ultra-compact form factor, ideal for breadboard prototyping and when space is limited. • Fully compatible with the environment and the Arduino code.	• Hardware specifications and performance similar to the Arduino Uno. • No native wireless communication interface.	22,00 €
Raspberry Pi Zero W	• High computing power with support for full-featured operating systems (Linux). • Ability to perform multitasking, complex data processing, and work with graphical user interfaces. • Excellent performance-to-price ratio for high-performance architectures.	• Greater complexity in software configuration and management. • High energy consumption.	18,76 €
ESP8266	• A Wi-Fi module with native integration and an exceptional price-to-performance ratio. • Fully compatible with the Arduino IDE ecosystem and toolchain. • The ideal solution for IoT (Internet of Things) and remote control applications.	• A limited number of programmable I/O pins (GPIO) are available for sensors.	7,99 €

Table 2: Available boards, with strenghts and limitations

Based on the information provided in [Table 2](#), the ESP8266 offers the best balance in almost every respect, which is why it was chosen from among all the hardware options listed. Compared to the Arduino UNO and Arduino Nano, it features native Wi-Fi connectivity; in fact, to integrate this functionality into either of those boards, an external module would need to be added, which would increase costs, physical size, and circuit complexity. Furthermore, the processor integrated into the ESP8266 which operates at 80 MHz and has 64 KB of RAM is much more powerful than Arduino's ATmega328P chip, which operates at 16 MHz and has only 2 KB of RAM. These features allow for significantly smoother data and memory management.

The Raspberry Pi Zero W includes Wi-Fi connectivity, but costs more than twice as much as the ESP8266. Furthermore, the ESP8266 executes the code loaded into memory directly. It takes less than a second to boot up and is immediately operational. The Raspberry Pi, on the other hand, must load an entire operating system (Linux) from a MicroSD card. It takes 20 to 40 seconds to boot up. Furthermore, if the power suddenly cuts out, the Raspberry Pi's SD card risks becoming corrupted, causing the system to freeze. In terms of power consumption, the ESP8266 uses very little power (just a few milliamps) and features advanced power-saving modes (Deep Sleep). It can run for months or years on battery power. The Raspberry Pi, with its complex processor, GPU (Broadcom VideoCore IV), and continuously running operating system, consumes much more power. A standard battery lasts only a few hours. Finally, as mentioned earlier, the ESP8266 is fully compatible with the Arduino IDE ecosystem and toolchain. This means it can be programmed using the same language, the same libraries, and the same logic as an Arduino Nano or Uno, without the steep learning curve or the complexity of managing a Linux operating system (as would be the case with the Raspberry Pi Zero W).

2.5.2 Sensors and actuators

The sensors that can be used to build this system are listed in [Table 3](#) and [Table 4](#), along with their pros and cons.

An analysis of the available actuators is provided in [Table 5](#).

The choice of sensors and actuators must be suited to the characteristics of the indoor home environment, which is characterized by the presence of a pet and, possibly, the residents of the home itself. Furthermore, to integrate immediately with the existing network, it is essential that the devices connect directly to the home Wi-Fi and operate completely autonomously.

As specified in **FR7**, the system must be able to extract data from the environment specifically, movement detection and, based on this data, extract visual information that will allow the system, in a subsequent phase, to identify and distinguish the pet from any other moving object (**FR6**).

To this end, after analyzing the available motion sensors, we selected the ultrasonic sensor (HC-SR04). This choice was based on the fact that it has a good range and is not affected by ambient lighting conditions; this means it works well at any time of day and is not dependent on whether the lighting system is on or off. Another factor is that, unlike other sensors, it does not detect objects based on their crossing a defined beam; rather, it is capable of detecting objects that pass through a circular volumetric sector with a measurement angle of 15–20°. Finally, among the available sensors, it is the least expensive option.

Moving on to the analysis of sensors designed to capture visual information, the selection focused on the ESP32- Camera OV2640 and OV7670 Camera Module. The Arducam Mini SPI (OV2640) was not included in the list of possible candidates because, despite having the advantage of a RAM buffer, it has the same resolution as the ESP32-Camera and costs about three times as much. Similarly, the Raspberry Pi Camera Module 3, while offering excellent resolution, is compatible only with the Raspberry Pi board and is also relatively expensive.

After analyzing the last two possible candidates, the ESP32-Camera OV2640 was selected. The initial evaluation of the OV7670 camera module revealed significant issues. First of all, the device requires interfacing with an external microcontroller such as the Arduino Uno. Furthermore, the lack of native network connectivity necessitates the integration of a dedicated Wi-Fi module, such as the ESP8266.

From a hardware perspective, it requires connecting so many pins that direct interfacing with a single board, such as the ESP8266 NodeMCU, is physically impossible.

The ESP32-CAM resolves all these issues. It integrates the camera and Wi-Fi onto the same PCB, and thanks to image compression, it enables nearly instantaneous data transmission.

As a deterrent, after evaluating the available actuators, the KY-0012 buzzer was chosen. This choice was based on the fact that, when an animal enters a restricted room, a simple vibrating actuator might not be effective enough to drive it out of the room (**FR8**). Acoustic devices are better suited for this purpose. Specifically, an active device was chosen because it emits a simple, continuous deterrent sound and requires fewer computational resources from the board's CPU.

Motion Sensor	Range	Strengths	Limitations	Cost
IR Emission + Receiver (KY-005 + KY-022)	20 cm – 400 cm	• Detection is limited to the line of sight between the transmitter and the receiver. • Not affected by changes in ambient temperature.	• An object that does not intersect the straight line is not detected. • Infrared radiation from the sun or incandescent light sources can blind the receiver.	11,99 €
Laser emit + Photoresistor (KY-008 + KY-018)	10 m – 15 m	• Instant response over medium-to-long range distances. • Millimeter-level precision at the breakpoint.	• Visible red dot (environmental disturbance). • The photoresistance varies with daylight.	10,80 €
IR Obstacle Avoidance Sensor Module (KY-032)	2 cm – 30 cm	• It does not require alignment with a receiver on the opposite side.	• Reduced range and an extremely limited operating range. • Dark surfaces absorb infrared light, reducing the effective range..	6,49 €
Ultrasonic sensor (HC-SR04)	2 cm – 450 cm	• It operates using acoustic pulses at 40 kHz, making it immune to changes in lighting. • Volumetric 3D scanning, with a measurement angle of 15–20°.	• The sampling rate is limited by the time it takes for the signal to travel there and back (500 ms). • The signal is attenuated by sound-absorbing materials.	4,40 €

Table 3: Available sensors, with strenghts and limitations

Camera-based Sensor	Strenghts	Limitations	Cost
Arducam Mini SPI (OV2640)	It includes 384 KB of buffer RAM and JPEG compression, relieving the microcontroller of the processing load and allowing you to take photos even on boards with limited memory, such as the Arduino UNO.	The SPI interface, running at only 8 MHz, makes it too slow for video streaming, limiting it to single frames.	39,98 €
Raspberry Pi Camera Module 3	- Reaches up to 120 fps at reduced resolution, making it ideal for fast object tracking or slow-motion footage. - Software-based autofocus. - Maximum resolution (4608 × 2592 pixels).	Not compatible with Arduino, NodeMCU, and ESP32; can only be used on single-board computers running Linux.	34,46 €
ESP32-Camera OV2640	- It features a built-in LED that can be used as a flash, support for OTA (over-the-air) firmware updates, and various sleep modes to conserve power. - It combines a standalone microcontroller, a camera (2 MP OV2640 sensor), Wi-Fi and Bluetooth connectivity, and a microSD card slot into a single module. - It does not need to be connected to other main units to operate.	- The resolution of the included OV2640 sensor is only 2 megapixels (UXGA resolution of 1622 × 1200 pixels). - As mentioned in the text, the basic version requires a separate module or external adapters for programming and power via USB.	12,89 €
OV7670 Camera Module	- It costs very little, making it perfect for low-cost prototypes. - In standby mode, it draws less than 20 μA to operate, making it perfect for devices that need to wake up and trigger only when necessary.	- Very low resolution and image quality (640 × 480 pixels). - No autofocus and poor performance in low-light conditions.	3,56 €

Table 4: Available Camera sensors, with strenghts and limitations

Actuator	Strenghts	Limitations	Cost
KS0450 Keyestudio Vibration Motor Module	• It emits no audible noise and produces a completely safe mechanical/tactile sensation. • The frequency and intensity of the vibration can be adjusted to create different pulses.	• Performance depends heavily on the material of the surface on which it is installed.	4,16 €
Buzzer Active Module (KY-0012)	• A simple logic pulse is all it takes to make them work. • They do not require the board (Arduino / ESP8266) to generate fixed-frequency square-wave signals, freeing up processing resources.	• They can produce only a single tone at a fixed frequency. • It is not possible to reproduce melodies, musical notes, or alarms with different frequencies.	1,95 €
Buzzer Passive Module (KY-006)	• They allow you to generate different frequencies, musical notes, melodies, and complex alarm sounds.	• The board to which it is connected must continuously generate the signal at the required frequency.	1,86 €

Table 5: Available actuators, with strenghts and limitations

2.5.3 Communication technologies and application protocols

The analysis of telecommunications technologies is presented in [Table 7](#), while the application protocols are listed in [Table 6](#).

In this regard, the nature of the system requires the use of secure, reliable communication technologies that comply with well-defined standards.

To select the appropriate communication technologies and application protocols, it is necessary to analyze the reference scenario and the requirements.

Analysis and choice of communication technologies

In our context, Wi-Fi (IEEE 802.11) proves to be the ideal solution compared to the other options analyzed, perfectly meeting the operational and functional constraints of the home environment. First of all, the operational environment consists entirely of indoor residential spaces where a pre-existing home Wi-Fi infrastructure is already in place. This eliminates the need to purchase and configure dedicated gateways or hubs (which are essential with technologies such as Zigbee, LoRaWAN, or Sigfox), allowing the nodes to communicate directly with the local network and the management server. This architecture directly satisfies the requirement for component replaceability and simplicity (**NFR2**). In fact, by relying on commercial, standard Wi-Fi modules, replacing a faulty device requires neither proprietary hardware nor complex re-pairing procedures with a central hub.

Furthermore, Wi-Fi fundamentally resolves the critical issues of traditional GPS, which loses accuracy or signal in indoor environments due to obstacles such as walls and ceilings. Compared to narrowband technologies such as Zigbee, LoRaWAN, Sigfox, or NB-IoT (designed exclusively to send very few bytes with minimal payloads), a Wi-Fi network offers the bandwidth and transmission speed needed to seamlessly handle a frequent stream of sensor data or the transmission of images which are essential because these images must be analyzed by the recognition algorithm (AI) to detect the pet's presence. Finally, the decisive factor is the drastic reduction in latency and the ease of cost-effective management. The system must, in fact, detect entry into a restricted room and activate the deterrent while simultaneously sending a notification to the owner within a maximum of 3 seconds of the event (**NFR4, FR8**). LPWAN technologies are not suitable for this purpose: in addition to having strict limits on radio transmission time (*duty cycle*), Sigfox severely limits incoming messages, while LoRaWAN and NB-IoT introduce unpredictable latencies and wake-up times that are incompatible with immediate action. On the other hand, traditional cellular solutions (such as 4G/5G or dedicated M2M SIM cards like Things Mobile) would entail recurring operating costs (data fees) and high energy consumption which are entirely unjustified in a home environment covered by a fixed-line network. Wi-Fi is therefore the only option capable of guaranteeing immediate two-way communication, high bandwidth, and real-time response without additional costs.

Analysis and selection of application protocols

The analysis of the application protocols was conducted by evaluating the nature of the exchanged data, the transmission frequency, the payload size, and the latency constraint imposed by the **NFR4**.

With regard to communication between ultrasonic sensor and camera modules, where the former serve to generate a trigger event for the latter, the device network employs an approach based on the **MQTT** protocol.

The MQTT protocol, which is based on the *Publish/Subscribe* model, designed for M2M communications and IoT environments, simple, and with low bandwidth consumption, represents

the ideal solution for this type of interaction.

Since the ultrasonic sensors module serve solely as triggers, the required payload is very small, ensuring extremely fast transmission and minimizing network bandwidth usage. Transmissions are *event-driven*: the signal is not transmitted continuously, but only when an entity (such as a person or a pet) crosses the door threshold. At that moment, the ultrasonic sensor node performs a publish, sending a light message to the broker, to which only the relevant cameras are subscribed, triggering the photo to be taken in real time with minimal latency. Finally, delivery reliability is guaranteed by the **QoS 1** level (*At least once*), which ensures that the trigger signal is received even in the event of temporary micro-interruptions in the local network.

However, when the image is subsequently transferred from the device to the backend server, the nature of the data changes: the image is a binary file that is larger than a simple string. For this operation, the **HTTP** protocol is used, which is ideal for sending individual files directly, without having to maintain a constantly open connection or go through an intermediary broker. Immediately after the photo is taken, the camera module sends the image to the server, which processes it using an artificial intelligence model for pet recognition, in order to distinguish pets from any other objects (**FR6**).

The activation signal is sent to the buzzer on the animal's device once again via **MQTT**. This command can be triggered automatically when the AI detects the animal in a restricted area (**FR8**), or manually when the administrator decides to activate it from the app (**FR13**). Since this is a simple control signal with a very small payload, using the broker allows the command to be sent asynchronously and immediately. This decouples the backend from the physical device and guarantees extremely fast response times, well within the 3-second latency limit (**NFR4**). In this case as well, **QoS Level 1** is used: guaranteed delivery prevents the buzzer command from being lost due to network issues, ensuring that activation occurs immediately.

Furthermore, MQTT allows for continuous monitoring of the connection status of each individual device (**FR14**): by leveraging the native *Last Will and Testament* (LWT) feature, the broker automatically detects any sudden disconnection of the node from Wi-Fi and instantly notifies the server.

Finally, an evaluation of the alternatives led to the decision to rule out CoAP and WebSocket. CoAP, to ensure message reliability over UDP, uses a retransmission mechanism based on timeouts and exponential back-off: if a packet is lost, the wait times between attempts increase exponentially, risking a violation of the 3-second latency constraint (**NFR4**). WebSocket, on the other hand, requires the establishment and maintenance of a permanent, full-duplex TCP channel (permanent link) between each device and the server. Keeping persistent connections open at all times would be inefficient for microcontrollers, since communication occurs only at specific times. The combined use of MQTT for lightweight messages and HTTP for photos is therefore the best choice for our system.

Analysis and choice of suitable user interfaces

To ensure interaction with the system, two complementary user interfaces have been developed: a web application and a Telegram bot, each designed for a specific use case.

The web application serves as the main browser-based interface. Through it, users can register (**FR1**) and log in (**FR2**). A user who creates a new Home environment automatically assumes the role of Administrator (**FR11**), gaining full control over the environment. From this platform, the Administrator can authorize or remove other users as simple viewers (**FR12**), configure the house map by managing rooms and their doors (**FR3, FR4**), and define the pet's profile (**FR5**). The web application also allows users to set the status of each room as allowed or restricted (**FR3**), view the pet's current location at any time (**FR9**), and access a section dedicated to metrics, violation statistics, and room occupancy data (**FR10**). Communication between the Web Dashboard and the server occurs exclusively via the **HTTP** protocol, utilizing **RESTful API** architectures for sending requests and managing data.

A Telegram bot has been integrated with the web application to handle quick interactions and send real-time notifications to mobile devices. After logging in using the credentials of the account already created on the web (**FR2**), the user can interact with the bot to request the animal's current location (**FR9**). The bot also serves as a direct channel for immediately receiving alerts if the animal enters a restricted area (**FR8**) and allows the Administrator to manually activate the remote acoustic deterrent (**FR13**). Finally, the bot interface monitors the status of the home network, sending timely notifications to the user in the event of a disconnection or when devices within the infrastructure regain their connection status (**FR14**).

From a structural standpoint, communication between the Telegram bot and the backend server occurs in two ways, always using the **HTTPS** protocol. When the server needs to send a message, it makes a direct **HTTPS** call to the Telegram API. The message reaches the user's mobile device via the chat identifier (*chat_id*), which is saved during login. When, on the other hand, the user sends a command from the chat, Telegram forwards it to the server via an **HTTPS** webhook. The bot first checks the profile permissions (distinguishing between Administrator and Viewer) and then calls the server's internal functions to read the database or send the command to the buzzer device (**FR13**).

Protocol	Strengths	Limitations
MQTT	• Three levels of QoS guarantees. • Outbound-only connections (bypasses firewalls/NAT without opening ports). • Disconnection management via LWT. • Publish/Subscribe model.	• Dependence on the central broker (Single Point of Failure). • Long and resource-intensive <i>wake-up time</i>: waking up from <i>sleep</i> requires re-establishing the TCP session.
CoAP	• RESTful architecture (easy integration with HTTP). • Low overhead (UDP) and piggybacked ACKs to conserve resources. • Designed for devices with minimal resources (< 10 KB RAM).	• The server cannot send data on its own (it requires polling). • Less mature and a smaller ecosystem compared to the standard Web. • Complex reliability management over UDP (trade-off between packet loss and congestion).
HTTP/HTTPS	• Client-server model with standard methods and URIs. • Multi-format support (JSON, XML, HTML). • TLS/SSL encryption (HTTPS) and universal compatibility.	• Not suitable for <i>event-driven</i> contexts (client-initiated communication). • Inefficient for real-time applications (frequent polling wastes bandwidth). • Difficulties with firewalls for bidirectional connections.
WebSocket	• Bidirectional, full-duplex communication over a single TCP connection. • Bandwidth/CPU efficiency and minimal message overhead (2 bytes per frame). • Transparent upgrade from HTTP (reuse of standard ports) and fallback to HTTP. • No ongoing connection management or coordination.	• Subject to browser security restrictions (Same-Origin Policy)

Table 6: Comparison of Communication Protocols

Technology	Strengths	Limitations
Wi-Fi (IEEE 802.11)	• High data transmission speed (data rate). • Widespread adoption and existing infrastructure. • Direct Internet connection without dedicated gateways.	• High power consumption (not suitable for small batteries). • Limited indoor coverage range (obstacles/walls).
Zigbee (IEEE 802.15.4)	• Native support for mesh networks (coverage extension). • Low power consumption. • High reliability in dense sensor networks.	• Very low data rate. • Requires a specific hub/gateway for IP conversion.
LoRaWAN (LoRa)	• Extremely long range (up to several kilometers). • Minimal power consumption (battery life of up to several years). • Excellent penetration in urban environments and through obstacles.	• Extremely low data rate (small payloads). • Not suitable for real-time or frequent transmission. • Requires a LoRaWAN gateway or infrastructure.
Sigfox	• Extremely low power consumption and low hardware costs. • A global LPWAN network managed without the need for proprietary gateways. • High signal penetration through obstacles.	• Severely limited data rate and number of daily messages. • Limited bidirectional (asymmetric) communication. • Dependence on Sigfox network coverage and subscription cost.
NB-IoT (Narrowband IoT)	• Excellent indoor and underground penetration. • Based on existing cellular infrastructure (3GPP standards). • Supports high device density per cell and low power consumption (PSM/eDRX).	• Requires a SIM card and a data plan from a mobile carrier. • Higher latency than standard 4G/5G. • Data rate limited to low speeds.
Cellular / M2M (Things Mobile)	• Global multi-carrier coverage via roaming. • Direct IP/Cloud connection without intermediate gateways. • High flexibility and support for device mobility.	• Recurring operating costs (SIM and data traffic). • Higher power consumption compared to dedicated LPWAN technologies.

Table 7: Comparison of Communication Technologies

3 Design phase

This section provides a detailed analysis of the system design, covering hardware, software, and the overall infrastructure. The goal is to define the technical specifications and show how the individual components interact with one another in a coordinated manner.

On the hardware side, the system integrates the sensors and actuator selected during the analysis phase, with the goal of making them work together while reducing complexity and meeting the project requirements. On the software side, the architecture has been designed in a modular fashion to ensure robustness and facilitate future extensions.

The general architecture is shown in Figure 1 and is divided into three macro-areas:

- **On Field:** It includes the physical devices installed in the home, namely the nodes with ultrasonic sensors for detection, the nodes with cameras for visual verification, and the wearable device for the animal equipped with a buzzer.
- **Server:** It serves as the central hub of the infrastructure. It includes the application backend, the artificial intelligence module for computer vision (*AI Vision*), the database (*MongoDB*), the broker (*HiveMQ*) and the tunnel management (*Ngrok*) for webhooks.
- **Client:** It includes the user interfaces, specifically the web application for system management and the Telegram bot for quick notifications and commands.

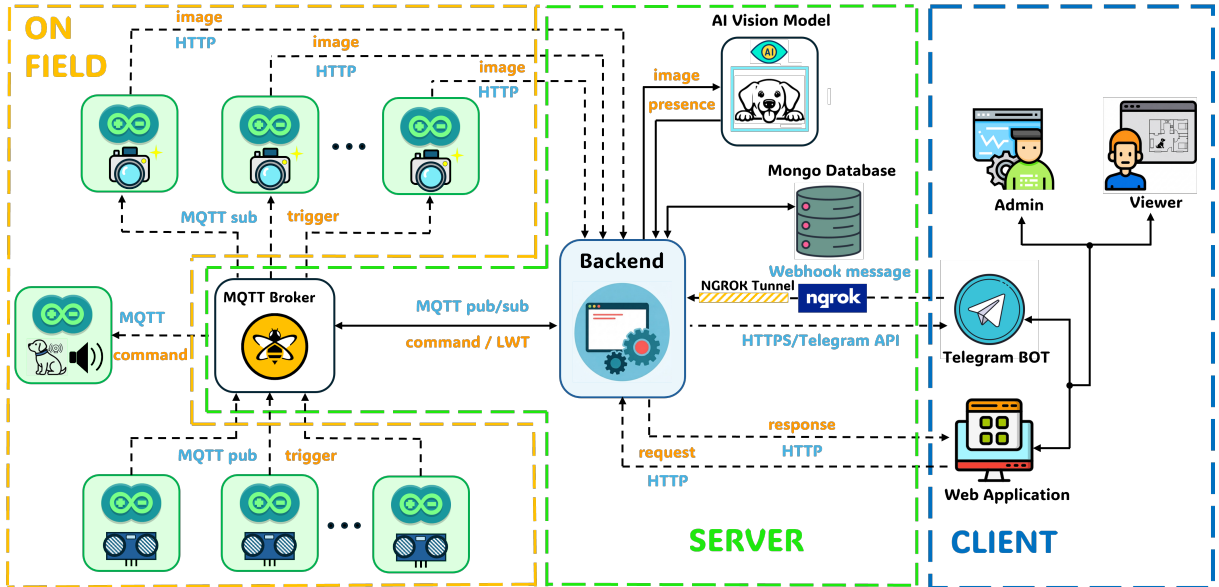


Figure 1: This figure provides an overview of the system architecture, where the dashed lines indicate Wi-Fi connections, while the solid lines represent interactions and data flow between components. The blue labels specify the application protocol used, while the orange labels indicate the type of data or command being exchanged.

A detailed description of each subsystem will be provided below, outlining their hardware specifications, software components, and interaction logic.

3.1 On-Field System Architecture

This section provides a detailed analysis of the system's *On-Field* components, focusing on the architecture of the sensing and actuation nodes, as well as on the logic and data management. More specifically, the electrical connections and pin mapping will be defined, and the interfacing between the ultrasonic sensors and the cameras will be discussed. In addition, *Software Flowcharts* will be developed for each of the *edge* nodes, defining the on-board decision-making logic as well as event handling. Finally, the data acquisition phase (*data collection*) by the sensors and the subsequent transmission phase (*data transmission*) to external systems, managed via **MQTT** or **HTTP** communications will be discussed in detail.

3.1.1 Passage Detection Module

Ultrasonic sensor nodes were used to detect when the pet moves from one room to another. To do this, they must be positioned in the door frames at a strategic height. This height must be chosen based on the animal's size. The diagram in Figure 2 illustrates the hardware architecture based on the NodeMCU ESP8266: in addition to managing the interface with the ultrasonic sensor, the device uses Wi-Fi connectivity to report passage events in real time via the MQTT protocol.

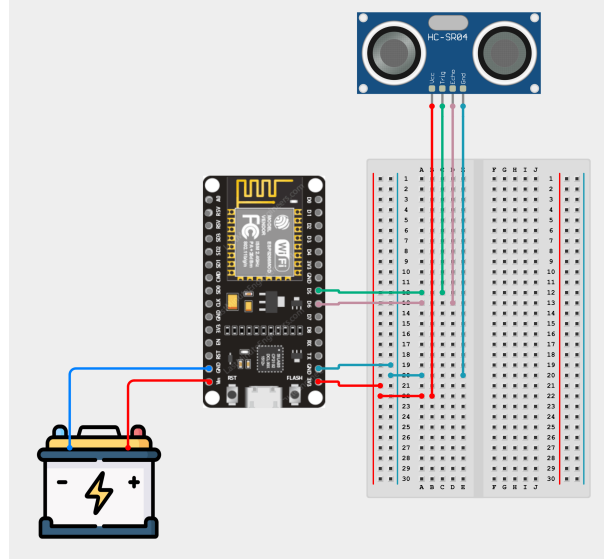


Figure 2: Wiring diagram for connecting the HC-SR04 ultrasonic sensor to the NodeMCU ESP8266 module, powered by an external power source.

From an operational standpoint, the microcontroller reads the distance at a fixed sampling frequency ($f_{\text{sensing}} = 2 \text{ Hz}$), corresponding to an interval of 500 ms. This ensures that the 40 kHz pulses emitted by the transmitter have time to travel and fade away before the next reading, preventing wave overlap and residual echoes. Differently, the transmission frequency is *event-driven*: the NodeMCU does not send data at regular intervals to the MQTT broker, but generates a *publish* only when the sensor node detects a complete passage through the door.

The Algorithm

Figure 3 shows a flowchart illustrating the logical sequence, starting with the network configuration (Wi-Fi and MQTT) and continuing through the measurement of distance using ultrasound and the activation of the trigger when an object passes through.

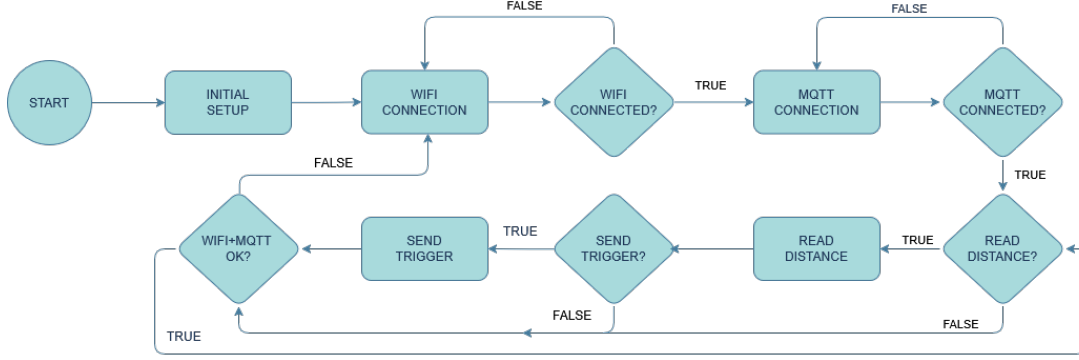


Figure 3: Software Flowchart of the Room-to-Room transition Algorithm

The logic flow begins with the initialization phase, during which the microcontroller configures pins D5 and D6 to manage the ultrasonic sensor's trigger and echo, and then connects to the Wi-Fi network. To ensure the integrity and confidentiality of data exchanged over the network, communication with the HiveMQ cloud broker takes place via an encrypted connection based on the Transport Layer Security (TLS) protocol on the dedicated port 8883. The TLS protocol, the successor to the SSL standard, ensures end-to-end encryption of the transmission channel, preventing the interception or manipulation of MQTT packets. This level of protection is complemented by application-level authentication using unique credentials (username and password), which is necessary for the broker to identify and authorize the client to publish messages.

To ensure continuous and robust operation, the system performs a preventive check of the status of network connections before executing any sensing operations. If Wi-Fi connectivity is lost, the firmware automatically initiates reconnection procedures. Similarly, the stability of the MQTT channel is constantly monitored: in the event of a disconnection, the device attempts to reconnect at regular intervals. Furthermore, to handle sudden failures or unexpected power loss, the system utilizes the MQTT protocol's *Last Will and Testament* (LWT) feature. This mechanism allows the broker to automatically report the connectivity status on a dedicated topic, indicating in real time whether the node is active or disconnected.

The NodeMCU sends a pulse to the Trigger pin and measures how long the Echo pin remains high, calculating the distance using the formula:

$$\text{distance} = \frac{\text{duration} \times 0.034}{2}$$

Where $0.034 \text{ [cm}/\mu\text{s}]$ represents the speed of sound in air (approximately 343 m/s at 20°C), while dividing by 2 accounts for the fact that the ultrasonic signal travels the distance between the sensor and the obstacle twice (round trip).

The detection algorithm does not use a fixed calibration, but continuously compares two consecutive readings while saving the previous measurement. At each cycle, it calculates the change in distance ($\Delta = \text{previous distance} - \text{current distance}$) and compares it to a threshold of 10 cm, set based on the animal's minimum width to avoid false positives caused by measurement noise:

- **Entry under the door ($\Delta \geq 10 \text{ cm}$):** The distance decreases by at least 10 cm, confirming that the animal has entered the sensor's range. The system sets the internal state to `petUnderDoor = true`.
- **Stationary ($|\Delta| < 10 \text{ cm}$):** If the animal remains stationary under the door or there are changes smaller than its minimum body size, the system ignores the noise and waits without sending any commands.
- **Crossing completed ($\Delta \leq -10 \text{ cm}$):** the distance increases again by at least 10 cm, indicating that the animal has exited the sensor's range and the door is clear again.

The “entry-exit” sequence confirms that the animal has completed its passage. Only at this point does the NodeMCU reset its state and send the command that triggers the cameras on the dedicated MQTT topic, causing them to take a photo.

The technical specifications and operating parameters are listed in [Table 8](#).

Parameter	Value / Specification
Microcontroller Module	NodeMCU (ESP8266)
Ultrasonic Sensor	HC-SR04 (Pin D5 TRIG, Pin D6 ECHO)
Sensing Frequency	2 Hz (Sampled every 500 ms)
Transmission Frequency	<i>Event-driven</i> (on passage event)
MQTT Trigger Topic (Pub)	home/<door_id>
Trigger Payload	Plain-text string (TURN_ON_CAMERA)
MQTT Status Topic (LWT)	home/<door_id>/state
Status Payload	Plain-text string (ONLINE / OFFLINE)
MQTT QoS Level	QoS 1 (<i>At least once</i>)
Broker Port / Security	Port 8883 (HiveMQ Cloud TLS + Auth)

Table 8: Technical specifications and network configuration of the Passage Detection Module.

3.1.2 Visual Acquisition Module

To capture visual data following movements between rooms, each room has been equipped with an ESP32-CAM module. To optimize power consumption and prevent network overload, the camera does not record continuously but responds only to trigger events generated by the ultrasonic sensor modules located at the access points.

For authentication purposes, as shown in Figure 4, the ESP32-CAM sends an HTTP POST request to the backend containing the room ID. This parameter alone is sufficient to identify the device, since only one camera is associated with each room. The server checks MongoDB to verify the room's existence and returns a JWT token, which the device stores to authorize subsequent image transmissions.

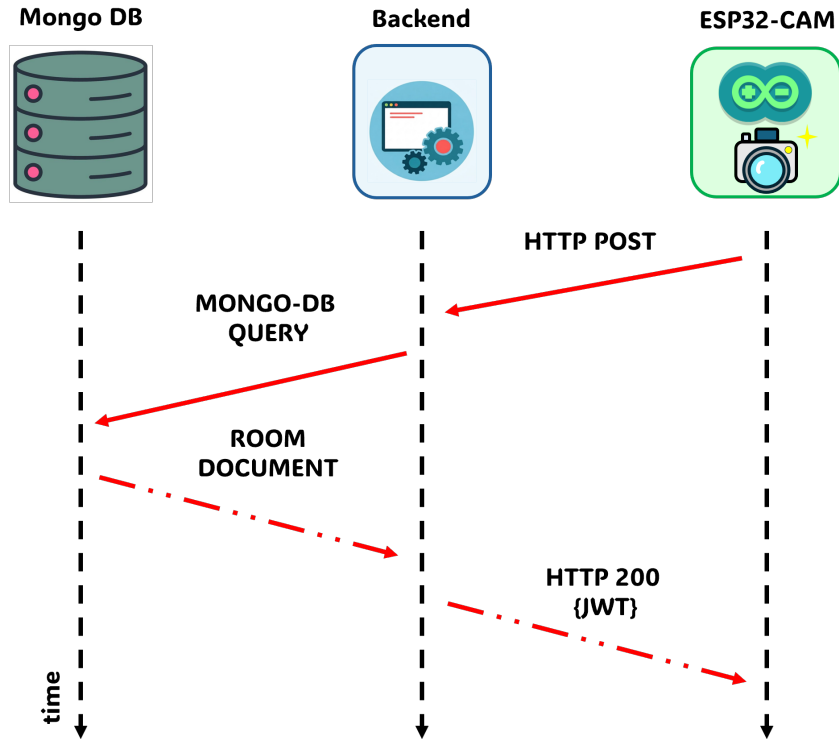


Figure 4: ESP32-CAM authentication procedure

The Algorithm

When the ESP32-CAM starts up, the *setup* phase initializes the camera sensor and initiates the Wi-Fi connection process using a blocking loop. Once network connectivity is established, the module performs the authentication process with the backend until it obtains the JWT token.

Next, the ESP32-CAM establishes a TLS-encrypted connection with the MQTT broker. Similar to what was described for the passage detection system, the LWT (*Last Will and Testament*) mechanism is configured to handle unexpected disconnections; the board then notifies its operational status and subscribes to the trigger topics associated with the room's access points.

Upon receiving a trigger signal from the ultrasonic unit, the board initiates the acquisition routine: it analyzes the ambient light conditions, activating the LED flash if necessary, captures

the frame, and transmits it to the backend via an HTTP POST request with multipart data authenticated using a JWT token. The entire operational flow is schematically represented in the flowchart in Figure 5.

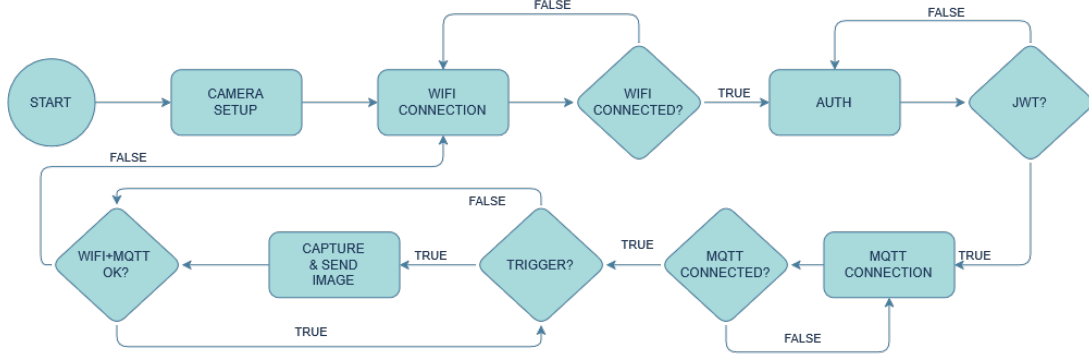


Figure 5: Software Flowchart of the ESP32-CAM module

The pinout of the ESP32-CAM module and the mapping of its I/O and power pins are shown in Figure 6, while the technical specifications are listed in Table 9.

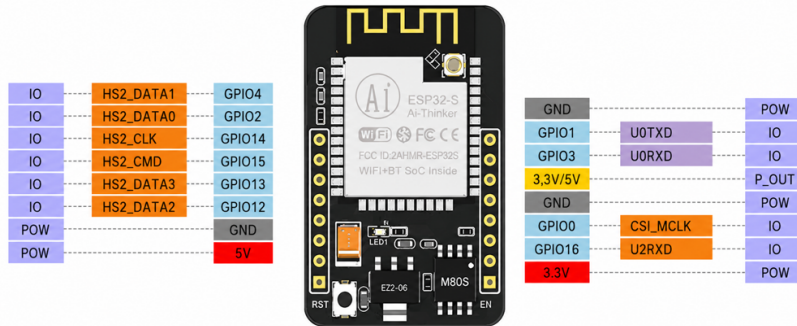


Figure 6: ESP32 schematic with pins

Parameter	Value / Specification
Module / Model	ESP32-CAM (OV2640 Sensor)
Sensing Frequency	Event-driven (Photo taken on trigger)
Transmission Frequency	Event-driven (HTTP POST on event)
MQTT Trigger Topic (Sub)	home/<door_id>
MQTT Status Topic (LWT)	home/<room_id>/state
MQTT QoS Level	QoS 1 (At least once)

Table 9: Technical specifications of the ESP32-CAM unit.

3.1.3 Acoustic Deterrence Module

The module, which includes the deterrent, is designed to discourage the animal from entering or staying in unauthorized rooms by emitting an audible signal via an active buzzer. Unlike other modules that are fixed in place, this device is designed to be portable and wearable, and must be integrated directly into the animal's harness or collar.

The circuit diagram shown in Figure 7 illustrates the hardware architecture based on the NodeMCU ESP8266 microcontroller: the signal pin of the active buzzer is connected to digital pin D0 of the NodeMCU. To ensure operation while the animal is moving, the entire system is powered by an external battery.

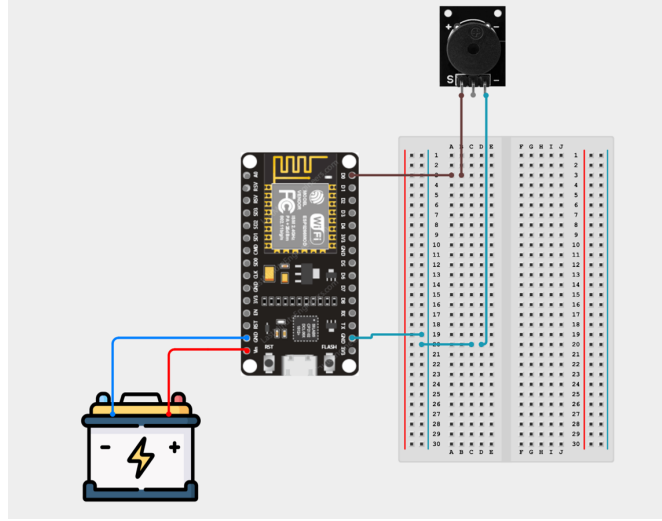


Figure 7: Wiring diagram for connecting the active buzzer module to the NodeMCU ESP8266, powered by an external battery.

As with the sensing and vision modules, the microcontroller connects to the HiveMQ cloud broker via Wi-Fi and a TLS encrypted connection on port 8883, following authentication with credentials. To detect unexpected disconnections or failures of the wearable node, the firmware configures the *Last Will and Testament* (LWT) mechanism on the dedicated status topic.

The algorithm

Figure 8 shows the flowchart of the module's logical sequence.

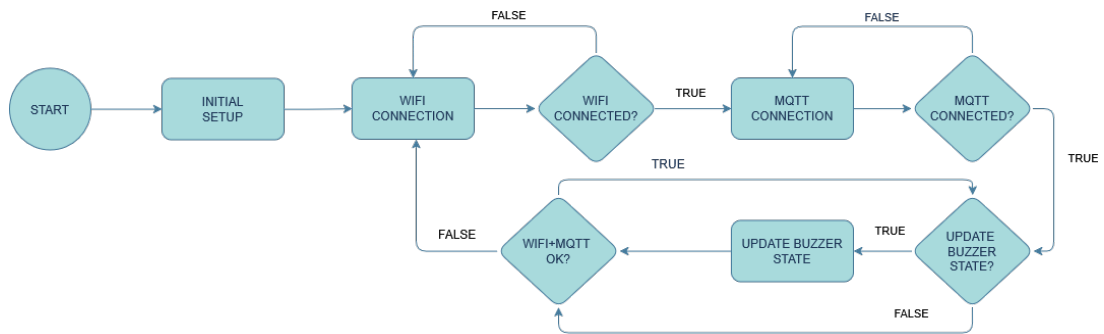


Figure 8: Software Flowchart of the Acoustic Deterrence Module

During the *setup* phase, the device configures the buzzer control pin, initially setting it to a *LOW* state. It then initiates a connection to the Wi-Fi network, connects to the MQTT broker, and subscribes to the topic dedicated to receiving commands.

During normal operation, just like the other modules, the system constantly monitors network connections: if a connection is lost, it initiates an automatic recovery procedure, thereby ensuring the device's operational continuity.

Next, the device establishes a TLS encrypted connection with the MQTT broker. As described for the other modules, the LWT (*Last Will and Testament*) mechanism is configured to handle unexpected disconnections; the board then notifies the status channel that it is operational and subscribes to the topic dedicated to receiving activation commands.

The unit continuously monitors commands sent via MQTT. The activation and deactivation of the acoustic deterrent correspond to two different scenarios:

- **Automatic Control:** The command is sent by the backend when the algorithm detects the animal's presence in a forbidden area. The shutdown signal is automatically transmitted as soon as the system detects that the animal has left the area.
- **Manual Control:** Activation is triggered directly by the user via the control interface. In this scenario, the audible alarm remains active until the next manual deactivation command is issued.

The technical specifications and operating parameters are listed in [Table 10](#).

Parameter	Value / Specification
Microcontroller Module	NodeMCU (ESP8266)
Actuator	Active buzzer (KY-0012)
Control Pin	D0 (GPIO16)
MQTT Command Topic (Sub)	home/sound
MQTT Status Topic (LWT)	home/buzzer/state
MQTT QoS Level	QoS 1 (<i>At least once</i>)

Table 10: Technical specifications of the buzzer unit.

3.2 Server

As shown in [Figure 1](#), the Server infrastructure encompasses the core components of the system that work together to manage data and requests.

At the heart of this layer is the Backend application, implemented using the Flask framework, which handles the system logic and provides the HTTP APIs. In addition to the Backend, the Server layer includes:

- **AI Model (Vision-based):** used to process images sent by the cameras and to verify the animal's presence.
- **Database:** dedicated to storing the system's status and data.
- **MQTT Broker:** manages publish/subscribe communication for exchanging commands and presence signals (*triggers*) with the *On Field* components.

3.2.1 Digital Twin Representation

The proposed system is based on the **Digital Twin (DT)** architecture. It adopts the **Resource VO** model for the virtual representation of the system's components. On field, the infrastructure consists of multiple *Things* dedicated to specific tasks (nodes for detecting passage through doors, camera nodes in rooms for capturing images, and a buzzer node attached to the animal). The choice of the Resource VO model is due to the fact that other types of VOs are not suitable for representing the components of our system.

The Object VO model is not suitable because it is designed to aggregate multiple capabilities within a single rigid structure. In our system, the Things perform distinct roles and are located in different positions:

- The Things placed at the doorways incorporate an ultrasonic sensor and monitor movement at the boundary between two adjacent rooms, making them relevant to both rooms;
- The Things dedicated to visual data collection are installed inside individual rooms;
- The Thing worn by the animal contains only the buzzer actuator.

Since a doorway is a boundary element connecting two distinct environments, and each room may have a variable number of doorways, it is neither possible nor conceptually correct to aggregate a "camera" Thing with a "doorway" Thing within a single VO Object. Attempting to do so would impose rigid and arbitrary associations. The Resource VO model overcomes this limitation by treating each component as an atomic and independent resource, decoupling the virtual representation of individual functionalities from the topology of the environment. Similarly, the Environment VO is designed to represent large-scale environments. This abstraction entails a loss of granularity (*loss of object-level detail*) that would prevent the precise identification of the source of data generated by a single node.

The main advantages of adopting the Resource VO model are:

- **Independent and decoupled resource utilization:** Each resource is modeled autonomously and exposes only its own atomic functionality. This ensures complete decoupling between the *Things*, allowing each node to manage its own events and data flows completely independently, without depending on the state of other devices.

- **Scalability (NFR1) and Replaceability (NFR2):** Treating each functionality as an isolated resource makes it possible to add, remove, or replace a single hardware node without having to restructure the digital representation of the other components or impact the backend logic.

Virtualization Layer

The decision regarding which types of objects to virtualize using the Resource VO model is guided by the need to monitor and manage, in a decoupled manner, both the operational status of the physical devices in the field and the dynamic status of the environment and the animal. Specifically, three types of Digital Replicas have been created (Figure 10):

- **Door DR:** Virtualizes each door. The need to model it arises from the requirement to monitor the connectivity status of the associated hardware node (the “Thing” that includes the ultrasonic sensor) and to decouple the detection of physical passage from the logic of capturing and sending the image.
- **Room DR:** Virtualizes each monitored room. It is used to track the operational status of the installed camera, the access permission level associated with the room (*allowed* or *forbidden*), the current occupancy status (presence or absence of the animal), and aggregated metrics on daily dwell time and the number of entries.
- **Pet DR:** Virtualizes the pet within the system. It monitors the connectivity status of the wearable device (the “Thing” that includes the buzzer), tracks the pet’s current location and updates it in real time (indicating the room it is in), and records behavioral metrics, including a history of violations of restricted zones and the duration of activated audible alerts.

Each virtual resource is instantiated from a standardized template (**Digital Replica**), defined in YAML format. The structure consists of:

- **common.fields:** contains unique identifiers (`_id`, `type`), descriptive profile information, and temporal metadata.
- **entity.data:** contains status data and historical and analytical metrics;
- **validations:** defines mandatory constraints on fields, restrictions on data types (numeric ranges), and the initialization of certain fields in the DR.

Table 11, Table 12, and Table 13 list the specific fields, their types, and the constraints associated with each type of DR.

Field	Description, Type, and Constraints
profile.name	String. Identifying name of the gate (required).
profile.description	String. Node description (default: “ <i>Smart Door with Ultrasonic Sensor</i> ”).
data.sensor.status	Enum (str). Operational status of the ultrasonic sensor [ONLINE, OFFLINE, UNKNOWN] (default: UNKNOWN).

Table 11: Fields and constraints specific to the Door DR.

Field	Description, Type, and Constraints
profile.name	String. Identifying name of the room (required).
profile.description	String. Textual description of the room.
profile.floor	Integer. Floor of the building with a range constraint of $[-1, 2]$ (required).
profile.permission_level	Enum (str). Access permission level for the animal [allowed, forbidden] (default: allowed, required).
data.status	Enum (str). Occupancy status of the environment [empty, occupied] (default: empty).
data.camera_status	Enum (str). Camera operational status [ONLINE, OFFLINE, UNKNOWN] (default: UNKNOWN).
data.last_entry_time	DateTime. Timestamp of the pet's last entry into the room.
data.occupancy_stats	List[Dict]. Occupancy history: date (datetime), daily_stay_duration_mins (float) and pet_entries_count (int).

Table 12: Fields and constraints specific to Room DR.

Field	Description, Type, and Constraints
profile.name	String. Pet's name.
profile.species	String. Animal species.
data.current_room	String. Name of the Room where the animal is currently located.
data.buzzer_status	Enum (str). Buzzer operational status [ONLINE, OFFLINE, UNKNOWN] (default: UNKNOWN).
data.last_buzzer_start_time	DateTime. Timestamp of the last buzzer activation.
data.daily_buzzer_stats	List[Dict]. Daily violation history: date (datetime), auto_violations_count (int), and auto_duration_mins (float).
data.learning_analytics	Dict. Dictionary reserved for the animal's behavioral analysis.
data.buzzer_state	Enum(str). Buzzer sound state [ON, OFF] (default: OFF)

Table 13: Fields and constraints specific to Pet DR.

In addition to virtualizing the system’s physical objects, the Virtualization Layer also manages the **User** entity. Although it does not represent a digital replica of an Object, the User shares the same modeling structure in YAML format to ensure consistency in data validation. This entity is essential for defining user profiles and mapping the corresponding access rights to virtualized environments, distinguishing between virtual environments managed with administrative privileges (`owned_homes`) and environments accessible with viewer-only permissions (`viewable_homes`).

The fields and constraints associated with the User entity are summarized in [Table 14](#).

Field	Description, Type, and Constraints
profile.username	String. Username for authentication (required).
profile.password	String. Hashed password (required).
entity.data.owned_homes	List[String]. Identifiers of the homes for which the user has administrator privileges (default: []).
entity.data.viewable_homes	List[String]. IDs of the homes for which the user has viewer permissions (default: []).
metadata.otp_code	String. Temporary numeric code generated for two-factor authentication (default: null).
metadata.otp_expires_at	DateTime. Expiration timestamp of the sent OTP code (default: null).
metadata.otp_verified	Boolean. Flag indicating that the second factor has been verified for the current session (default: false).

Table 14: Fields, types, and constraints of the User entity.

Service Layer

The Service Layer represents the processing layer of the architecture, positioned between the Virtualization Layer and the Application Layer. Each home environment is associated with one and only one Digital Twin, whose structure is based on a combination of multiple Digital Replicas and multiple Services (*Multiple DRs and Multiple Services*), as illustrated in [Figure 10](#). While the individual Digital Replicas (DRs) serve as passive virtual representations, the services are responsible for extending their operational capabilities. Through the processing of application logic, the Service Layer coordinates and integrates these different replicas within the home’s single dynamic Digital Twin, ensuring synchronization between the real-world environment and user applications. The following section analyzes the four main services that make up this layer, illustrating their operational logic and respective responsibilities.

Database Service

The Database Service is responsible for storing and retrieving system data, such as user data and Digital Replica data, and is located within the Server block of the overall architecture ([Figure 1](#)). MongoDB, a document-oriented NoSQL solution, was chosen as the database. It was chosen primarily for its schema flexibility, which allows for the management of data with different structures that vary over time without the rigid constraints of relational databases.

Before performing any operation, the service consults a central registry to determine which collection to save the information in and to verify that the data format is correct. It also manages

the connection to the database and provides all the main functions: saving new Digital Replicas, searches, document updates (automatically inserting the modification date), and deletions.

The service also manages user permissions, distinguishing between the home administrator and viewers. If a home environment is deleted from the system, the module performs a cascading cleanup by removing that reference from all associated user profiles. Finally, it handles user registration and the retrieving of their profile, ensuring that each username is unique.

Specifically, the data managed by the service is organized into four main collections in MongoDB, each dedicated to a specific application domain:

- **digital_twins:** stores documents related to the virtual environments managed by the system (each residence is associated with a DT). Each document contains the unique identifiers (`_id`), the name and description of the environment, as well as the list of Digital Replicas linked to it.
- **digital_replicas:** collects documents related to the Digital Replicas of field objects. The collection is heterogeneous and includes three distinct types (door, room, and pet), storing their operational status, profile settings, and the history of behavioral and analytical metrics.
- **users:** contains the profiles of users registered in the system. For each user, the system stores login credentials (with the password stored as a secure hash), the list of environments in which the user acts as an administrator (`owned_homes`), and the list of those accessible with view-only permissions (`viewable_homes`), as well as metadata for two-factor authentication (OTP code and expiration date).

MQTT Service

The MQTT Service manages the connection with the MQTT broker and synchronizes the status of IoT devices in real time with their corresponding Digital Replicas stored in the database. It is located in the Server block of the general architecture ([Figure 1](#)) and its task is to monitor the connection status of devices in the various environments and to alert users if one or more devices fail, disconnect, or require user intervention (**FR14**). HiveMQ was chosen as the broker, configured with secure authentication via username and password and a TLS encrypted connection. Upon startup, the service establishes the connection and subscribes to the topics related to device status ([home/+state](#)).

When a status update is posted to a topic, the service reads the DR identifier in the topic and updates the status in the database via the Database Service. The system checks sequentially whether the ID belongs to a room, a door, or a pet, updating the fields that track the connection status of the various Objects (`*_status`) accordingly.

To avoid false alarms or continuous notifications caused by brief disconnections, the service uses two asynchronous timers:

- **Disconnection Handling (OFFLINE):** Upon receiving a disconnection signal (handled via the Last Will and Testament mechanism), a 30-second timer is started. When the timer expires, the status of all devices in the home is checked. If any are still disconnected, a summary alert is sent to the user via Telegram, specifying whether the entire system has gone offline.

- **Restoration (ONLINE):** When a device comes back online, a 10-second monitoring timer starts. After 10 seconds, the service checks whether the entire home network has stabilized and, only if all devices are connected again, sends a system restoration notification via Telegram.

Pet Detection Service

When a camera sends an image to the server, the service uses the **yolo11l** model (Figure 1) to verify the presence of the pet (FR6). The choice of this model is mainly justified by its accuracy, as it allows the animal to be recognized even when it is small or partially hidden by furniture, while maintaining smooth processing on the server.

The flowchart in Figure 9 illustrates the logical flow on which this service is based (FR7). For the sake of completeness, the initial block relating to the detection of the room transition, the event from which the entire process starts, has also been included in the diagram. If the model's inference does not identify the animal, the service makes no changes and leaves the states of the *room* and *pet* type DRs associated with that DT in the database unchanged. If the animal is detected, the system retrieves the room of origin by reading the `current_room` field stored within the pet's DR. If the room in the image coincides with the current `current_room`, the processing ends without making any changes. This situation could occur when an entity other than the pet triggers the passage, such as a person or a cleaning robot.

If, on the other hand, a move to a new room is detected, the service manages the transition by updating the fields of the Digital Replicas involved:

- 1) **Digital Replica of the previous room:** using the room of origin extracted from the Digital Replica of the *pet*, the service retrieves the record of the old room and updates its `status` field from *occupied* to *empty*, stopping the count of the time spent there.
- 2) **Digital Replica of the animal:** the `current_room` field is updated by overwriting it with the name of the new room in which it has just been detected.
- 3) **Digital Replica of the new room:** the service updates the `status` field of the new room from *empty* to *occupied* and stores the entry time in the `last_entry_time` field.

Concurrently with these status updates, the system records data related to the time spent in the rooms and any violations, which will then be processed in detail by the dedicated statistics services (*Room Statistics Service* and *Pet Statistics Service*) as shown in Figure 10.

If the new room the animal has moved into has a forbidden permission level, the service publishes an MQTT message sending ON to the topic `home/sound`. This command activates the acoustic deterrent (buzzer) and, simultaneously, send an alert notification to the user via Telegram (FR8).

The deterrent remains active to dissuade the animal until it leaves the restricted area. As soon as the animal moves and is detected in an allowed room, the service publishes the OFF message on the same `home/sound` topic to deactivate it.

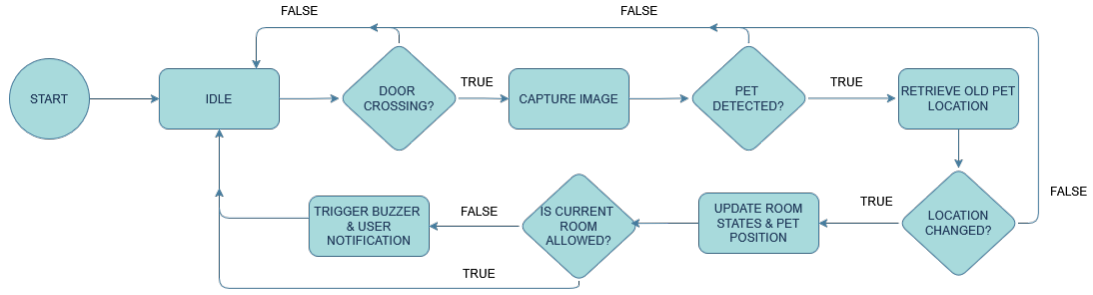


Figure 9: Flowchart of the *Pet Detection Service* showing the logical sequence from the passage trigger to image verification (FR6), followed by the update of room/pet states (FR7) and the potential activation of the buzzer (FR8).

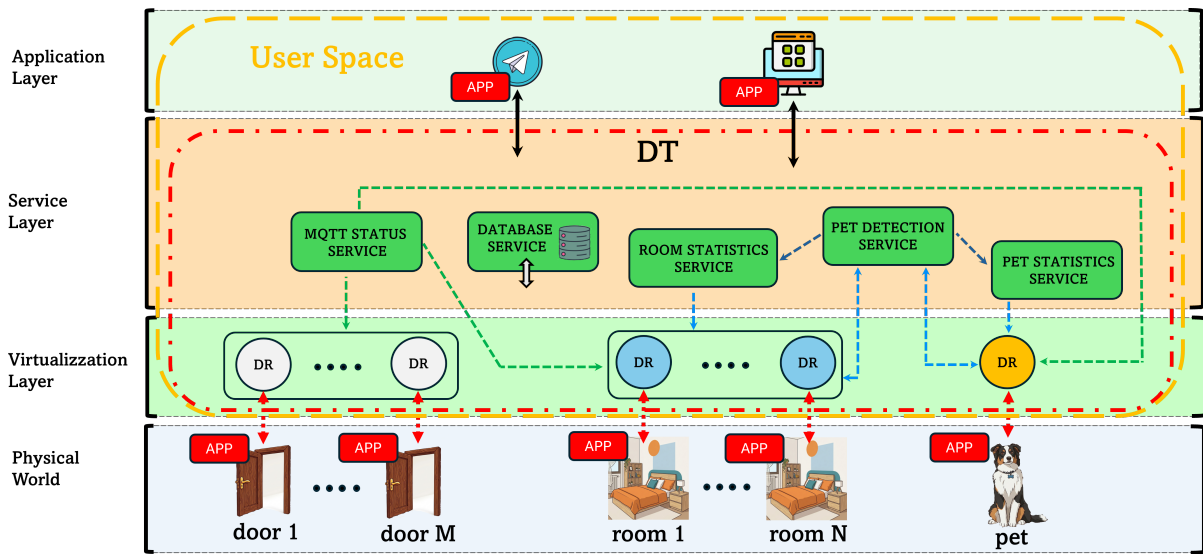


Figure 10: Layered Architecture of the Digital Twin: division into *Physical World*, *Virtualization Layer* (Digital Replicas), *Service Layer* and *Application Layer* (User Space).

Room Statistics Service

The Room Statistics Service is responsible for tracking and aggregating daily metrics regarding individual room occupancy. Unlike persistent historical logs, this service focuses exclusively on the twenty-four-hour cycle: accumulated counters automatically reset at midnight (UTC) to ensure the statistics reflect only the current day.

Specifically, the service processes the received data (duration of stay in minutes and number of entries to be recorded) by performing the following operations:

- **Chronological continuity check:** analyzes the record in the Digital Replica of the room where the animal was detected to verify whether tracking data for the current day already exists.
- **Counter update or reset:**
 - If a record for the current day already exists, the service increments the existing counters for the number of entries (`pet_entries_count`) and the cumulative stay duration (`daily_stay_duration_mins`).
 - If it is a new day (or the first execution), the old tracking data is replaced by initializing a new data structure containing the current date and the current values.
- **Synchronization with the Digital Replica:** the updated statistics state is finally overwritten in the database within the `occupancy_stats` array of the corresponding room's Digital Replica.

Pet Statistics Service

This service is designed to monitor, aggregate, and evaluate a pet's habits regarding adherence to allowed zones. Its primary purpose is to transform raw data collected from movement tracking such as unauthorized entries or the activation/deactivation of the deterrent into clear indicators of the training program's effectiveness over time.

For each recorded event, the service captures and updates three key parameters in a daily log:

- **Violation Count:** The total number of times the animal entered forbidden rooms during the day.
- **Active Duration of the Deterrent Module:** The total time (in minutes) the deterrent device was active to prompt the animal to leave the room.
- **Average Reaction Time:** A metric calculated as the ratio of the total deterrent activation duration to the number of violations recorded during the day ($\text{Duration}/\text{Violations}$).

Monitoring of the metrics listed above is limited to the last 30 days. When a new violation is recorded, the service checks the date of the most recently saved record. If an entry for the current day already exists (normalized to midnight UTC), the system updates the data for the ongoing session by adding the detected minutes and violations. If no records exist for the current day, a new record is created and added to the beginning of the history.

In addition to tracking metrics related to violation history, the service performs a comparative analysis by dividing recent history into two adjacent time windows (up to a maximum of 7 days each specifically, the current week and the previous week). The system calculates the percentage change in both total violations and average reaction time using the following formula:

$$\text{Variation \%} = \left(\frac{\text{Recent Value} - \text{Previous Value}}{\text{Previous Value}} \right) \times 100$$

Based on the variation thresholds set at $\pm 10\%$, the service automatically assigns a qualitative label describing the animal's learning status. These are listed in [Table 15](#).

Learning State	Condition, Criterion, and Behavioral Significance
Learning (Improving)	Reduction of more than 10% in violations or reaction times compared to the previous period. The animal is responding well to the deterrent and demonstrates an understanding of the established boundaries.
Trained (No Violations)	The violation count has been reset for both the recent and previous periods. The training was successful, as the animal consistently respects the boundaries and avoids entering forbidden rooms.
Regressing / Disobedient	An increase of more than 10% in violations or reaction times. This implies an increase in the animal's tendency to enter and remain in forbidden rooms.
Stationary	Variations within the $\pm 10\%$ threshold. Performance is stable, with no significant improvements or deteriorations.
Insufficient Data	History of less than 2 days of recording or absence of comparable parameters. Insufficient temporal baseline for reliable data analysis.

Table 15: Learning stages and assessment criteria of the Pet Statistics Service.

Upon completion of the calculation cycle, the updated 30-day historical data array and the structure containing analytical trends and learning status are saved to the database, directly updating the animal's Digital Replica record (`dr_type="pet"`).

3.3 The Flask application

The Flask application serves as the orchestration core of the entire backend architecture. Chosen for its lightweight and flexible structure, it handles HTTP request routing and the overall coordination between the platform's modules.

Upon startup, the server bootstraps the infrastructure. It establishes a connection to the database, loads the formal definitions for instantiating Digital Replicas (DRs), and activates services for managing the Digital Twin and its associated virtual replicas. Furthermore, as previously mentioned, it integrates the computer vision module dedicated to pet detection.

The server also coordinates two complementary communication channels: on the one hand, an MQTT infrastructure for real-time, asynchronous data exchange with the Objects distributed throughout the home; on the other, a Telegram interface based on webhooks and a secure tunnel, providing the user with an interactive channel for remote control and the immediate receipt of notifications.

API Organization and Security

The REST routes provided by the server are structured into distinct modules based on functional scope. The complete set of endpoints offered by the system is summarized in [Table 16](#).

To protect home resources and critical operations, the infrastructure implements security mechanisms based on access control and authentication using JSON Web Tokens (JWT). With the exception of initial registration and authentication procedures, every request sent to the endpoints requires a valid access token to ensure both the requester's identity and the integrity of the exchanged data ([Figure 11](#)).

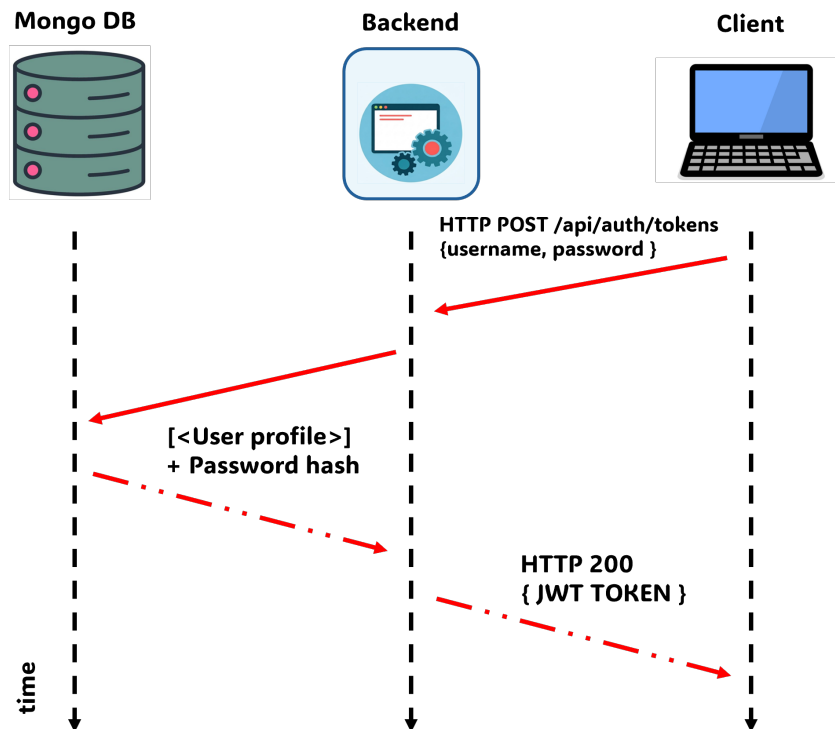


Figure 11: Schema of the authentication procedure. If the authentication succeeds, the client obtains the secure token.

Endpoint	Method	Parameters	Description
/api/users — Users			
/	POST	username , password	Registration of a new user.
/<user_id>	DELETE	—	Account deletion and cascading data removal.
/api/auth — Auth			
/tokens	POST	username , password	Login and JWT token issuance.
/tokens	DELETE	—	User logout.
/otp	POST	—	Send OTP code via Telegram.
/otp/checks	POST	otp_code	Verify OTP code correctness.
/api/dt — Digital Twins			
/	POST	name , description	Creation of a new home.
/	GET	—	List of the user's homes.
/<dt_id>	DELETE	—	Removal of home and related replicas.
/<dt_id>/viewers	POST	viewer_username	Add observer (Viewer).
/<dt_id>/viewers/ <viewer_id>	DELETE	—	Remove observer.
/<dt_id>/management	GET	—	Home entity inventory.
/<dt_id>/statistics	GET	—	Room occupancy statistics.
/<dt_id>/pet/statistics	GET	—	Pet statistics and trends.
/api/dr — Digital Replicas			
/<dt_id>/replicas/ <dr_type>	POST	name, [dr fields]	DR creation (room/pet/door).
/<dt_id>/replicas/ <dr_type>/<dr_id>	DELETE	—	DR deletion.
/<dr_id>/tokens	POST	—	IoT device authentication.
/<dt_id>/rooms/ <room_id>/telemetry	POST	image (file)	Upload photo for pet detection.
/<dt_id>/replicas/ <dr_type>/<dr_id>	PUT	[updated fields]	DR update.

Table 16: Summary of the system's REST API routes.

3.4 Client

The Client layer comprises the system's user interfaces. To ensure intuitive interaction (**NFR3**) and protection against unauthorized access (**NFR5**), the system provides two complementary interfaces: a web application and a Telegram bot. System users are divided into two main roles: the Administrator (automatically assigned to the person who creates the environment **FR11**), who has full control over configurations, and the Viewer, who has read-only access.

3.4.1 Web Application

The web application serves as the main interface, accessed via a browser. In accordance with requirements **FR1** and **FR2**, access to the platform is strictly contingent upon user registration and subsequent authentication, a process illustrated in [Figure 12](#), which allows users to securely create a new account or log in using existing credentials.

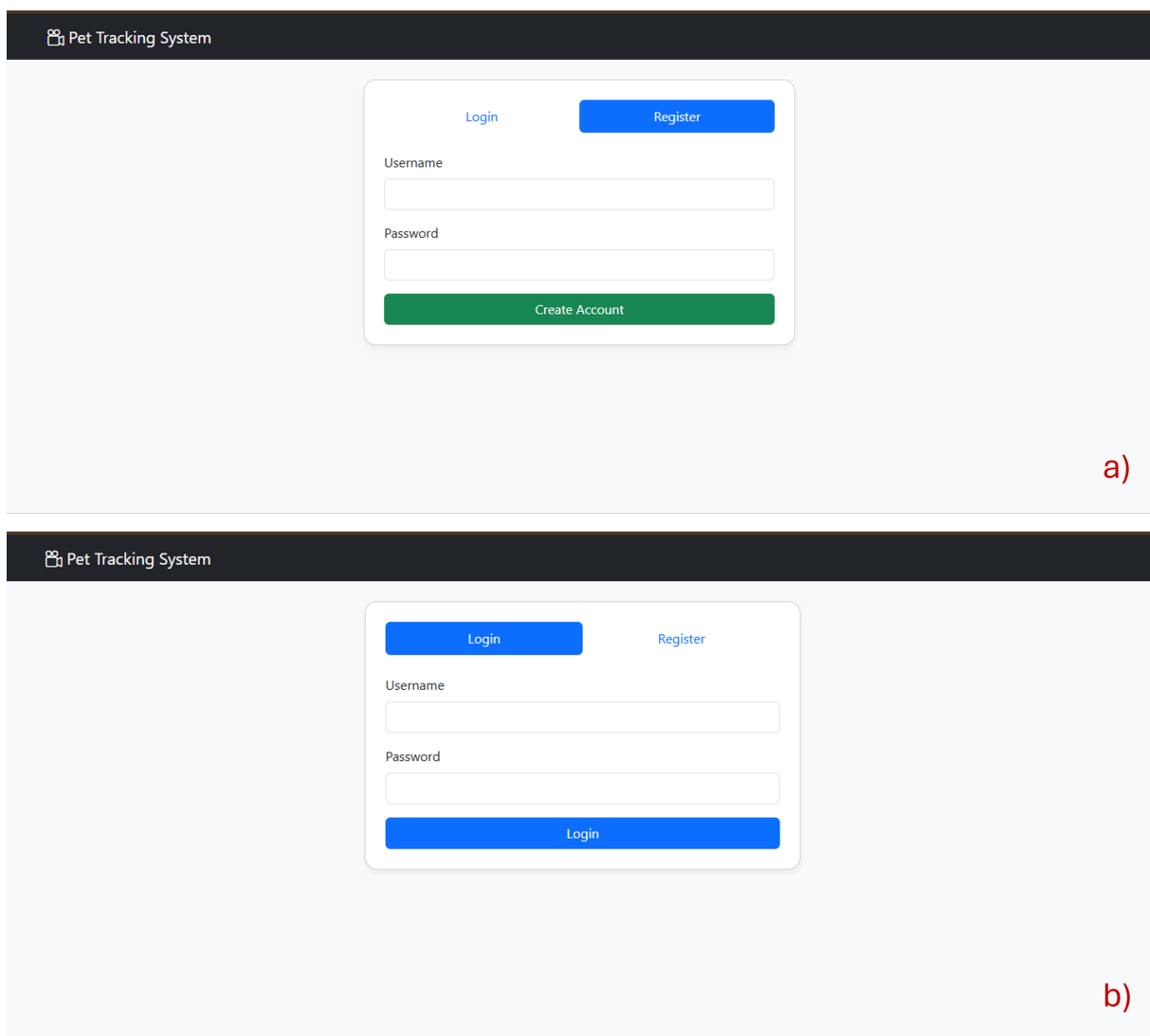


Figure 12: Screenshots of the registration (a) and login (b) pages for the web platform.

Once logged in, the user can proceed to create and configure their home environment. As shown in Figure 13, the Administrator defines the house map by entering room profiles, specifying their floor and permission level to determine allowed or restricted areas (FR3), then register doors (FR4) and defining the profile of the pet (FR5).

a) Step 1: Create Home Environment

The first screenshot shows the 'Environment Setup' page with the title 'Step 1: Create Home Environment'. It features two input fields: 'Name' (with placeholder text 'Home Name') and 'Description' (with placeholder text 'Brief description'). Below these fields is a blue button labeled 'Create Home'. The page header includes the 'Pet Tracking System' logo and a 'Logout' button.

b) Step 2: Add Components

The second screenshot shows the 'Environment Setup' page with the title 'Step 2: Add Components'. It contains three sections for adding components:

- Create Room:** Includes a 'Name' field (placeholder: 'e.g. Kitchen'), a 'Floor' field (value: '0'), a 'Permission' dropdown (value: 'Forbidden'), and a green '+' button.
- Create Door:** Includes a 'Name' field (placeholder: 'e.g. Front Door') and a green '+' button.
- Create Pet:** Includes a 'Name' field (placeholder: 'e.g. Fido') and a yellow '+' button.

Below these sections, a summary table shows the components added so far:

Rooms	Doors	Pets
None yet	None yet	None yet

A blue button labeled 'Go to Dashboard' is at the bottom of the form. The page header includes the 'Pet Tracking System' logo and a 'Logout' button.

Figure 13: Environment setup workflow on the web interface: a) Creating the home environment; b) Adding components (rooms, doors, and only a pet).

All home virtual environments created by the user or to which the user has been added as viewer are listed on the *Your Homes* Dashboard (Figure 14). In this section, specific visual labels allow you to immediately distinguish the user's role for each home (Administrator or Viewer).

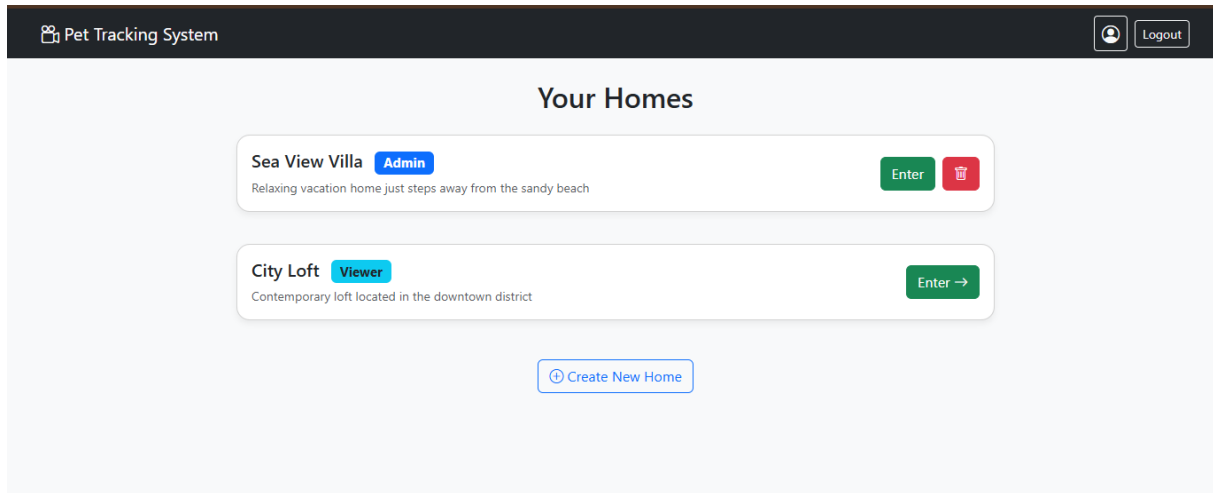


Figure 14: The "Your Homes" dashboard displaying the list of available home environments, their assigned user roles (Admin or Viewer), and action buttons.

Selecting a specific home, the user accesses the main dashboard of the system (Figure 15). Regarding the monitoring features, the platform allows users to check the real-time location of their pet within the home upon request (FR9), while also displaying the current status of the buzzer activation. In this section, only the Administrator can view the *Management* button, which allows access to the Home Management section.

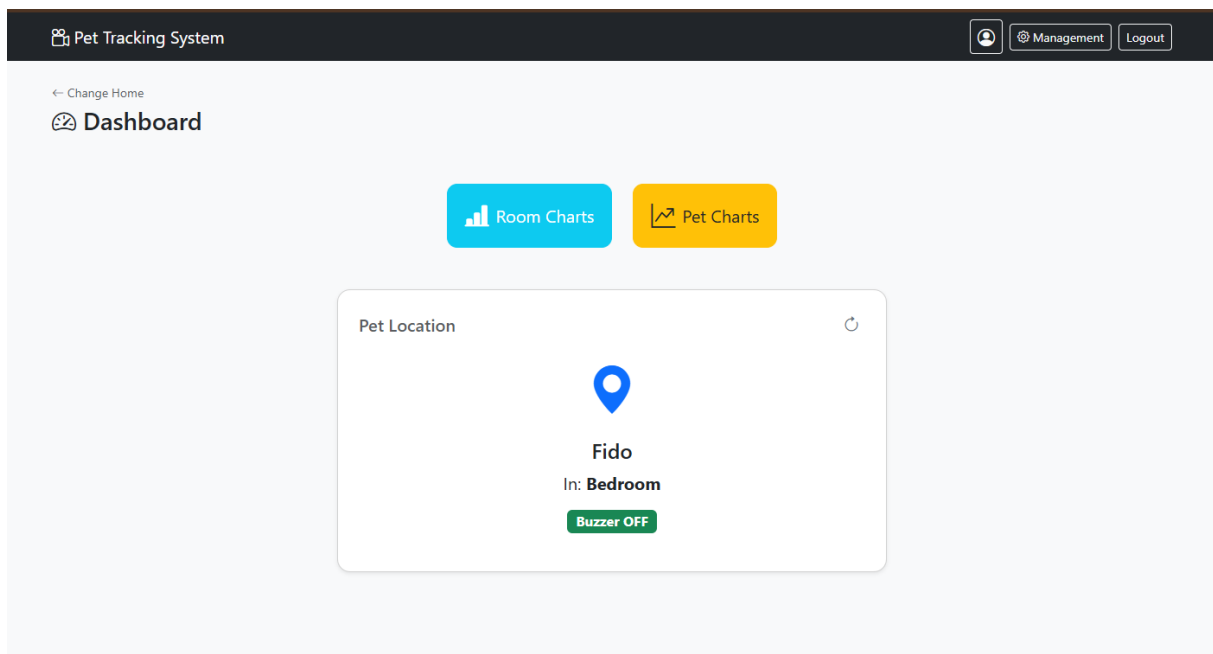


Figure 15: Main dashboard for real-time monitoring of the pet's location and the buzzer's status.

Through the Home Management section (Figure 16), the Administrator can modify the home's architecture at any time, update access permissions for each room, and add or remove Viewer profiles dynamically (FR12).

Pet Tracking System Management Logout

← Back to Dashboard

Home Management

Home ID Show ID

Rooms

Kitchen	allowed	Allowed	✓	✕
Living room	allowed	Allowed	✓	✕
Bedroom	forbidden	Forbidden	✓	✕
Office	allowed	Allowed	✓	✕
Bathroom	allowed	Allowed	✓	✕

Name e.g. Bedroom

Floor 0 Permission Allowed ✓

Doors

Kitchen/Living room door ✕

Name e.g. Back Door +

Pet

Fido ✕

Name e.g. Rex +

Viewers

stefano ✕

Username e.g. john_doe +

Figure 16: Home Management interface reserved for the Administrator to manage rooms, access, and Viewer profiles.

In order to maximize the overall security of the system (**NFR5**), access to the most sensitive sections of the platform (such as Home Management, managing one's data on the User Profile page shown in [Figure 17](#), and permanently deleting an account or home environment) is protected by a multi-factor authentication system.

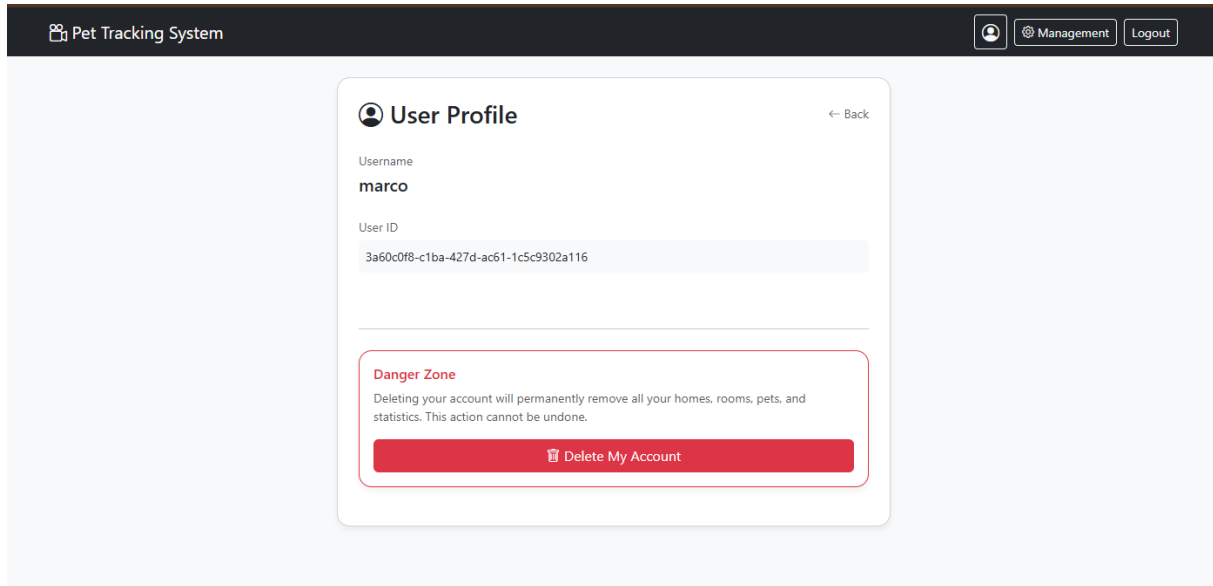


Figure 17: User profile section.

This security procedure requires the entry of a one-time password (OTP) sent directly to the user's mobile device via the Telegram bot, as shown in [Figure 18](#).

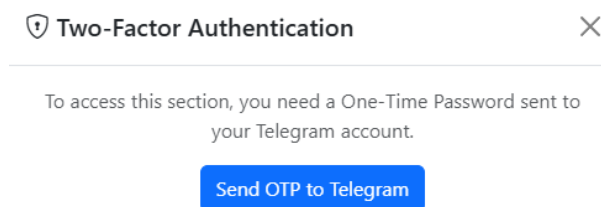


Figure 18: OTP verification request for access to critical sections.

Finally, to ensure long-term monitoring and support behavioral analysis, there is a section dedicated to statistics and logs (**FR10**). The system provides graphical dashboards showing the physical occupancy of rooms (**Figure 19**), tracking daily entries and dwell time for each room.

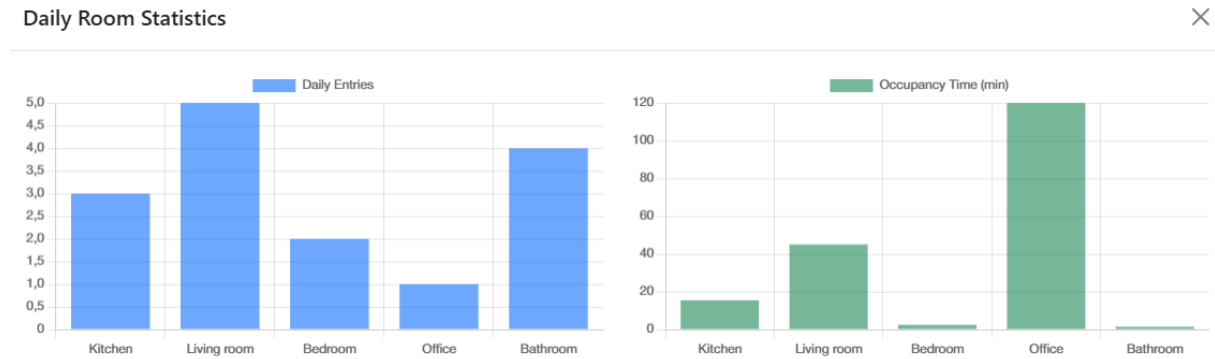


Figure 19: Daily room occupancy statistics, including the number of entries and length of stay.

In addition to spatial occupancy, the platform provides detailed metrics on the pet's learning progress (**Figure 20**). Through time-series graphs, the system analyzes the history of violations of restricted zones and the average buzzer activation time, providing analytical feedback on the effectiveness of the training program.

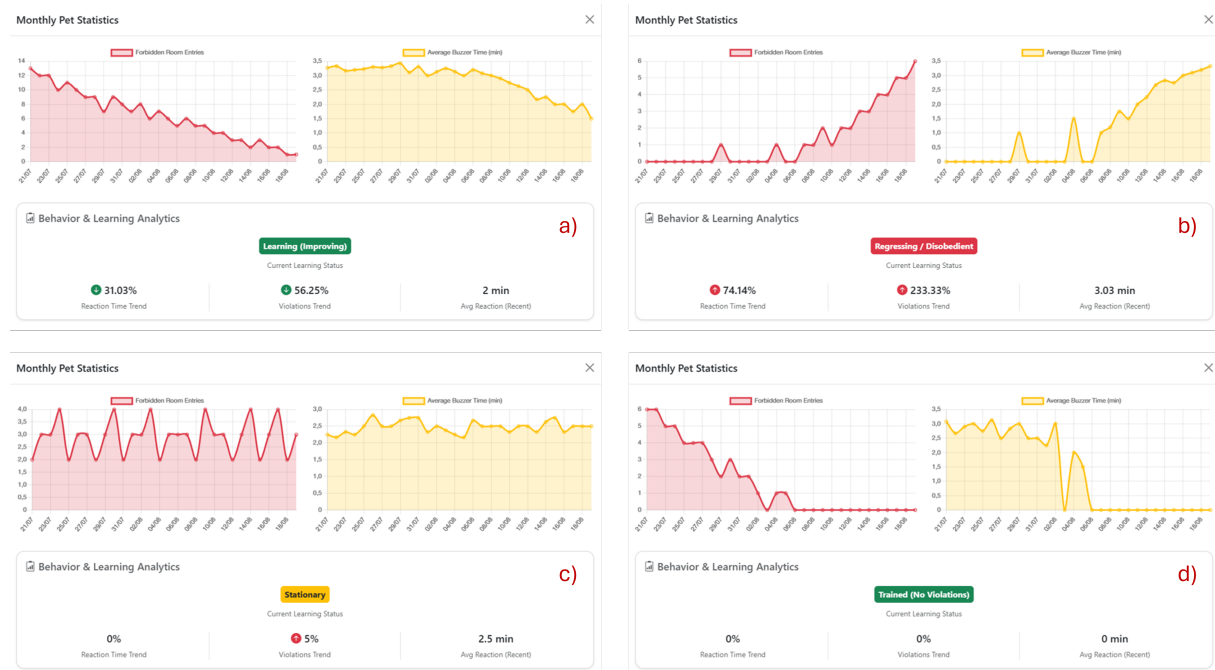


Figure 20: Behavioral metrics and learning progress of the pet based on the history of violations.

3.4.2 Telegram BOT

The Telegram bot complements the web application for managing quick interactions and sending push notifications to mobile devices, an important aspect for ensuring compliance with the system's strict latency limits, set at a maximum of 3 seconds (**NFR4**). The bot's startup and introduction screen is shown in [Figure 21](#). The user authenticates with the bot using the same credentials as the account previously created via the web application (**FR2**). Once logged in, the bot serves not only as a monitoring tool but also as an additional layer of security (**NFR5**), as it is responsible for delivering the OTP codes needed to unlock critical actions within the web application, thereby preventing accidental or malicious changes. In addition to this security function, the bot acts as a direct channel for alerts, sending immediate notifications as soon as the animal enters a restricted area (**FR8**). This tool is also essential for monitoring the status of the home infrastructure, promptly notifying the user of any disconnections, malfunctions, or restarts of IoT devices (**FR14**). In addition to passively receiving information, by interacting with the chat, the user can issue a direct command to request the pet's current location (**FR9**) or to remotely activate and deactivate the acoustic deterrent (**FR13**).

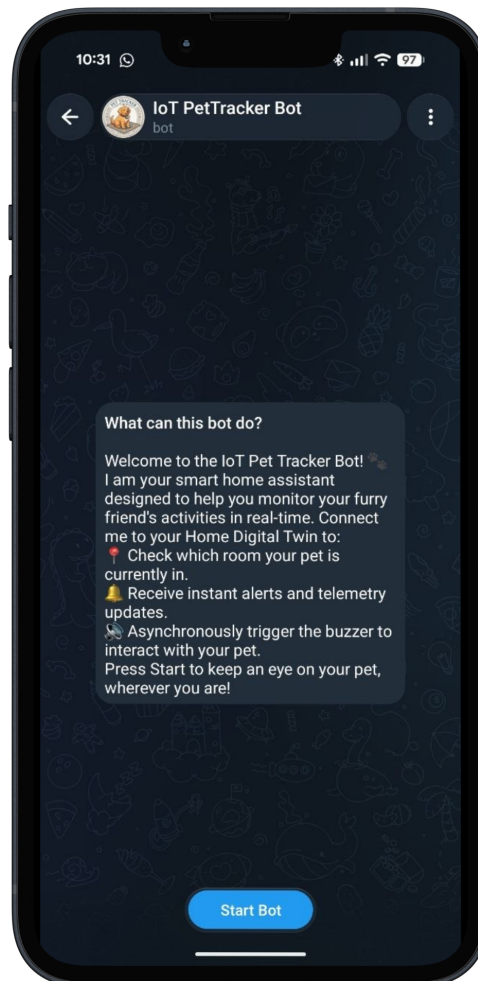


Figure 21: Initial presentation screen of the Telegram bot upon opening the chat.

Communication between the local Flask backend and Telegram relies on webhooks. Since Telegram requires a public HTTPS URL to deliver incoming HTTP POST requests, Ngrok is used to map the local server port to a temporary public endpoint. When a user interacts with the bot, Telegram sends the message to the Ngrok URL, which routes it through a secure tunnel to the local Flask application. Received requests are then forwarded to specific function handlers based on the following logic:

- **Authentication and Security (login_handler, logout_handler):** Access to features is blocked until the user links their account using the `/login <username> <password>` command (FR2). To ensure security and privacy (NFR5), the bot attempts to immediately delete the message containing the password from the chat. The user can then select the home to manage using an interactive *Inline Keyboard*, as shown in Figure 22.

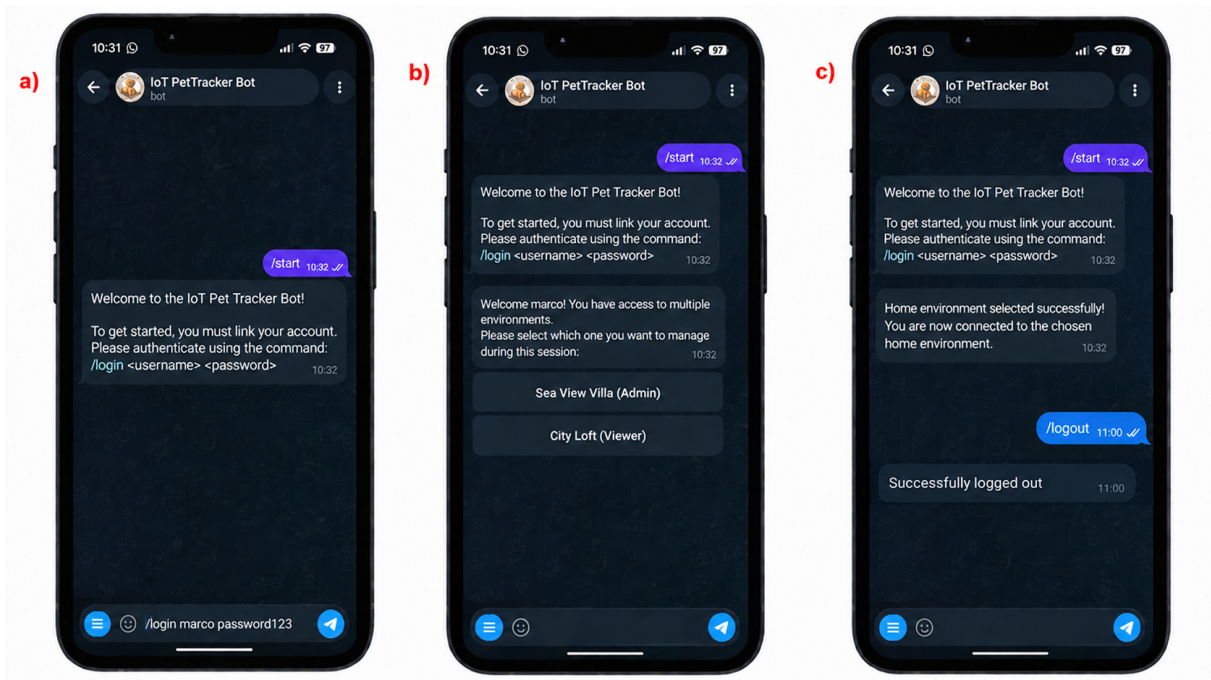


Figure 22: Telegram Bot interaction flow for user authentication and session management: a) Initial welcome message and login prompt; b) Environment selection based on user permissions; c) Session logout confirmation.

- **Location (locate_handler):** Handles the `/locate` command. The bot queries the system to retrieve the room currently occupied by the pet (**FR9**). (Figure 23 a).
- **Asynchronous Deterrent Control (buzzer_handler):** The `/buzzer` command allows manual activation and deactivation of the deterrent (**FR13**). The handler applies strict controls (Figure 23 b): it verifies that the user has the Administrator role (Role-Based Access) and evaluates spatial constraints. If the pet is in a restricted area, the bot refuses to process the manual command, since the actuator is already controlled automatically by the security system.

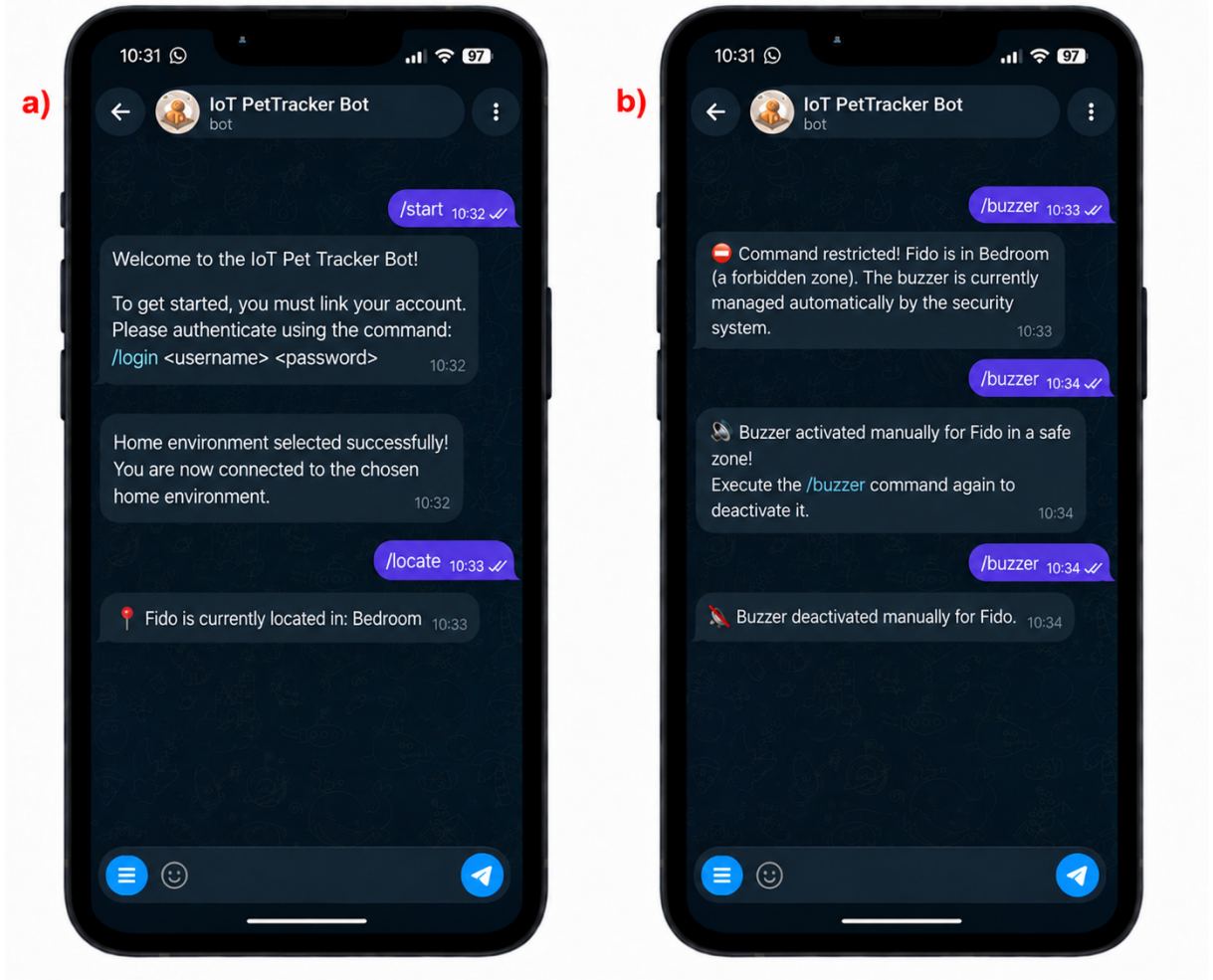


Figure 23: Telegram interface illustrating location retrieval and asynchronous deterrent control: (a) account session setup and pet location query via the `locate_handler`; (b) multi-state operation of the `buzzer_handler`, demonstrating spatial restriction rejection in restricted areas (top), followed by manual activation (middle) and deactivation (bottom) in safe zones.

- **Startup and Help (start_handler, help_handler, echo_handler):** These provide welcome messages and a list of available commands to guide the user during their first few uses (Figure 24). If text is entered that is not recognized as a valid command (echo_handler), the bot intercepts it and suggests consulting the help menu.

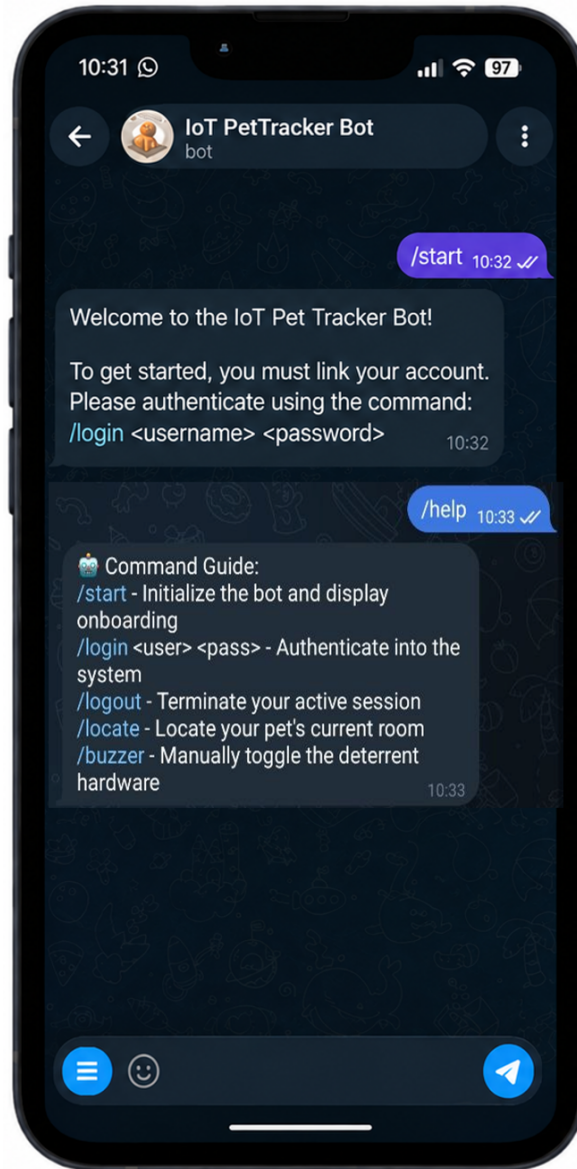


Figure 24: Bot response to the `/start` and `/help` commands for initial setup and guidance.

The bot does not merely respond to commands, it also enables the receipt of *real-time* notifications. This proactive channel is used to send intrusion alerts (Figure 26 d), OTP codes for critical web authorizations (Figure 25), and device disconnection alerts (Figure 26 a, b). Intelligently, when the hardware network is restored after a failure, the bot removes the old (offline) error messages from the chat, replacing them with a single restoration (online) message to keep the user interface clean and readable (Figure 26 c).

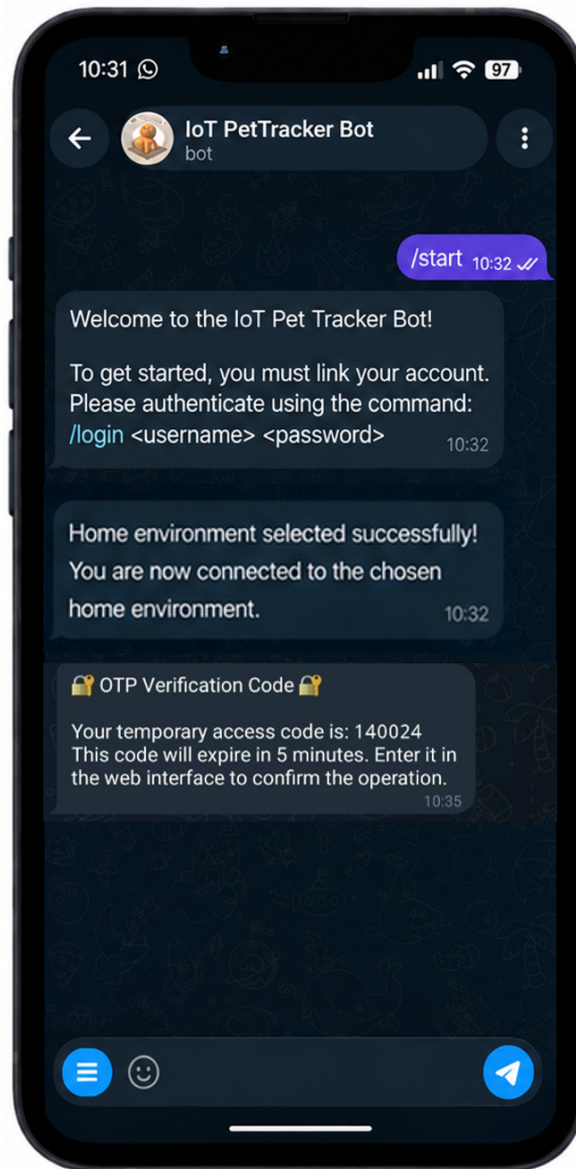


Figure 25: Receiving a temporary OTP code generated by the system to unlock a critical action in the web interface.

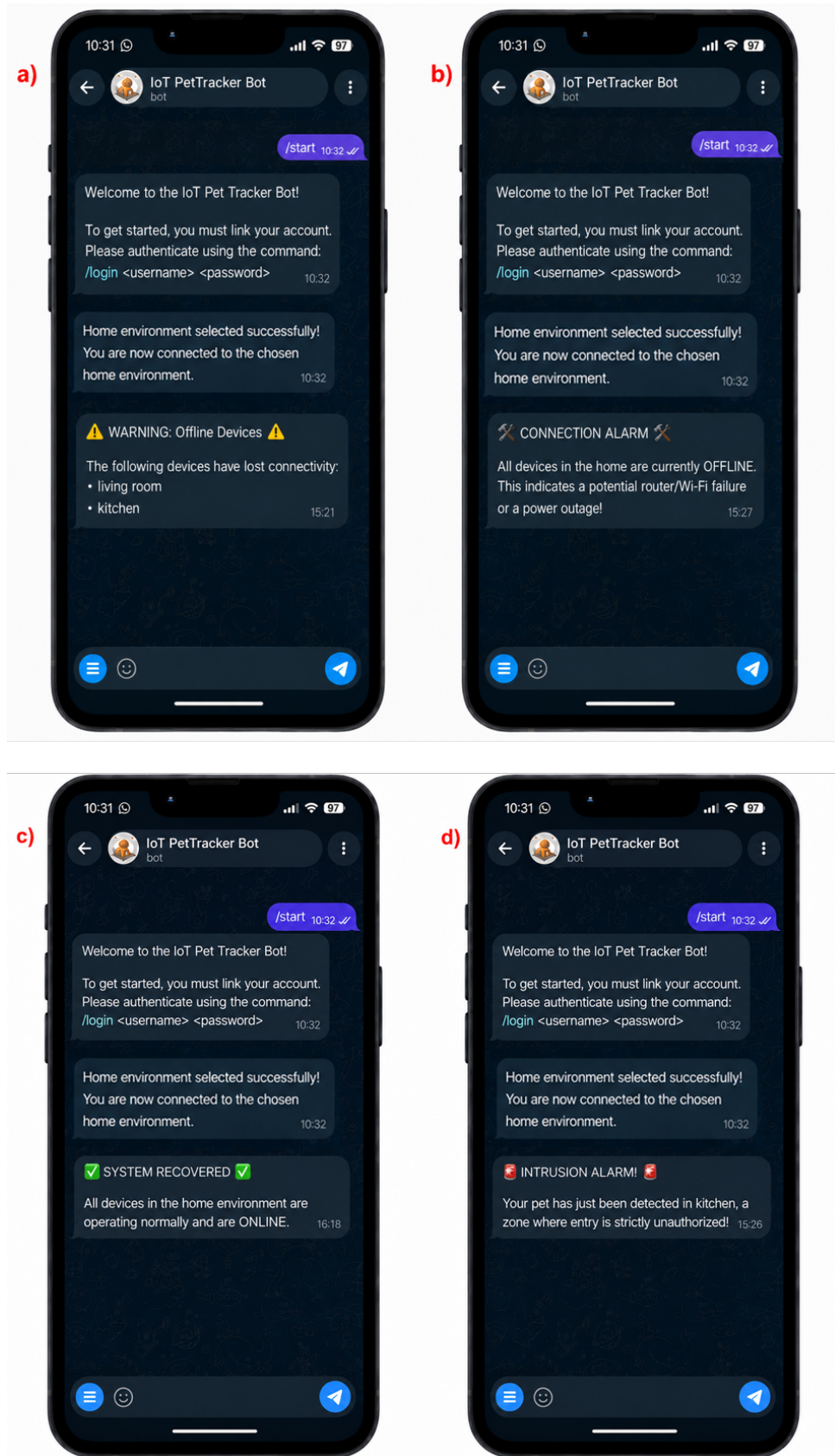


Figure 26: Real-time system notifications and alerts delivered via Telegram: (a) partial connectivity loss indicating specific offline devices; (b) total connection alarm representing home network failure or power outage; (c) system recovery confirmation after connectivity restoration; (d) intrusion alarm triggered by pet entry into a restricted area.

The non-functional requirement **NFR4** was validated using the following method:

The validation was conducted by performing a series of $N = 10$ crossing tests under real-world conditions. The overall end-to-end latency T_{total} was modeled as the sum of two distinct time components:

$$T_{\text{total}} = T_{\text{local}} + T_{\text{network}}$$

1. **Local processing latency (T_{local}):** Represents the elapsed time between the detection of the entry into the room and sending the actuation command to the buzzer. It was calculated using the difference between the message timestamps extracted directly from the **MQTTX Web** client:

$$T_{\text{local}} = t_{\text{publish}}(\text{home/sound}) - t_{\text{publish}}(\text{home/<door_id>})$$

2. **Network and notification latency (T_{network}):** Represents the time interval between the activation of the buzzer and the arrival of the notification on the user's smartphone. This latter time was measured using a standard stopwatch:

$$T_{\text{network}} = t_{\text{telegram}} - t_{\text{sound}}$$

By summing the two terms, the total latency T_{total} measures the interval from the initial physical event to the final delivery:

$$T_{\text{total}} = t_{\text{telegram}} - t_{\text{publish}}(\text{home/<door_id>})$$

The results obtained from the measurements across the 10 samples show a total End-to-End latency equal to:

$$T_{\text{total}} = 2.771 \pm 0.088 \text{ s}$$

Since the measured total latency remains consistently below the 3-second threshold, **NFR4** is fully satisfied.

References

- [1] N. Alavala, H. Chen, R. Singhal, and A. Dhekne, “Pettrack: Tracking pet location and activity indoors,” in *Proceedings of the 1st International Workshop on Hot Topics in Systems and Networking for Smart Animals (OdySys '22)*, New York, NY, USA: Association for Computing Machinery, 2022. DOI: [10.1145/3539489.3539587](https://doi.org/10.1145/3539489.3539587) [Online]. Available: <https://doi.org/10.1145/3539489.3539587>
- [2] A. Pangestu et al., “Pet tracking system using gps with android-based geofencing method,” in *2023 8th International Conference on Information Technology and Digital Applications (ICITDA)*, 2023, pp. 1–6. DOI: [10.1109/ICITDA60835.2023.10427488](https://doi.org/10.1109/ICITDA60835.2023.10427488) [Online]. Available: <https://doi.org/10.1109/ICITDA60835.2023.10427488>
- [3] Z. Tang, H. Hile, S. Bajracharya, and R. Jurdak, “Pettracker – pet tracking system using motes,” *University of California, Irvine / University of Washington*, Jan. 2005. [Online]. Available: https://www.researchgate.net/publication/237368760_PetTracker_-_Pet_Tracking_System_Using_Motes
- [4] Adirobot, *Arduino nano board: Features and technical specifications*, https://win.adirobot.it/arduino/arduino_nano/scheda_arduino_nano.htm, 2010.
- [5] RS Components, *Raspberry pi zero w datasheet*, https://www.reichelt.com/it/it/shop/prodotto/raspberry_pi_zero_w_v1_1_1ghz_512mb_di_ram_wlan_bt-256438?utm_source=display&utm_medium=rsf-foundation&, 2012.
- [6] Make It, *Nodemcu details and specifications*, <https://www.make-it.ca/nodemcu-details-specifications/>.